

Controle microprogramado

16.1 Conceitos básicos

- Microinstruções
- Unidade de controle microprogramada
- Controle de Wilkes
- Vantagens e desvantagens

16.2 Sequenciamento de microinstruções

- Considerações sobre projeto
- Técnicas de sequenciamento
- Geração de endereços
- Sequenciamento de microinstruções do LSI-11

16.3 Execução de microinstruções

- Taxonomia de microinstruções
- Codificação de microinstruções
- Execução de microinstruções no LSI-11
- Execução de microinstruções no IBM 3033

16.4 TI 8800

- Formato da microinstrução
- Microsequenciador
- ULA com registradores

16.5 Leitura recomendada

PRINCIPAIS PONTOS

- Uma alternativa para unidade de controle por hardware é a unidade de controle microprogramada, na qual a lógica da unidade de controle é especificada por um microprograma. Um microprograma consiste de uma sequência de instruções em uma linguagem de microprogramação. Trata-se de instruções que especificam micro-operações.
- Uma unidade de controle microprogramada é um circuito lógico relativamente simples que é capaz de (1) sequenciar pelas microinstruções e (2) gerar sinais de controle para executar cada microinstrução.
- Assim como em uma unidade de controle por hardware, os sinais de controle gerados por uma microinstrução são usados para causar transferências de registradores e operações de ALU.

O termo *microprograma* foi criado por M. V. Wilkes no começo dos anos 1950 (WILKES, 1951^a). Wilkes propôs uma abordagem para design de unidade de controle que era organizado e sistemático e evitava a complexidade de uma implementação embutida. A ideia intrigou muitos pesquisadores, mas parecia inviável porque iria requerer uma memória de controle rápida e relativamente cara.

A tecnologia de microprogramação foi revista na revista *Datamation* na sua edição de fevereiro de 1964. Nenhum sistema microprogramado estava em grande uso naquele tempo e um dos artigos (HILL, 1964^b) resumizou a

visão popular daquela época de que o futuro da microprogramação “é um tanto nebuloso. Nenhum dos grandes fabricantes mostrou o interesse na técnica, embora aparentemente todos a tenham analisado”.

Esta situação mudou consideravelmente poucos meses depois. O System/360 da IBM foi anunciado em abril e todos os modelos, exceto os maiores, eram microprogramados. Embora a série 360 tenha antecedido a disponibilidade da memória ROM semicondutora, as vantagens de microprogramação eram suficientemente atraentes para IBM fazer esse movimento. A microprogramação se tornou uma técnica popular para implementar a unidade de controle dos processadores CISC. Nos últimos anos a microprogramação começou a ser menos usada, mas permanece como uma ferramenta disponível para os projetistas de computadores. Por exemplo, conforme já vimos no Pentium 4, as instruções de máquina são convertidas em um formato parecido com RISC e a maioria delas é executada sem o uso de microprogramação. Entretanto, algumas das instruções são executadas usando a microprogramação.

16.1 Conceitos básicos

Microinstruções

A unidade de controle parece um dispositivo bastante simples. Mesmo assim, implementar uma unidade de controle como uma interconexão de elementos lógicos básicos não é uma tarefa simples. O projeto deve incluir a lógica para sequenciamento por meio de micro-operações, execução de instruções, interpretação de *opcodes* e decisões tomadas com base em flags do ALU. É difícil projetar e testar tal peça de hardware. Além disso, o projeto é relativamente inflexível. Por exemplo, é difícil alterá-lo se alguém quiser adicionar uma nova instrução de máquina.

Uma alternativa usada em vários processadores CISC é implementar uma unidade de controle microprogramada.

Considere a Tabela 16.1. Além de usar os sinais de controle, cada micro-operação é descrita em notação simbólica. Esta notação parece-se de forma suspeita com uma linguagem de programação. De fato, é uma linguagem, conhecida como **linguagem de microprogramação**. Cada linha descreve um conjunto de micro-operações ocorrendo ao mesmo tempo e é conhecida como uma **microinstrução**. Uma sequência de instruções é conhecida como um **microprograma** ou *firmware*. Este último termo reflete o fato de que um microprograma é uma ponte entre hardware e software. É mais fácil projetar um *firmware* do que um hardware, mas é mais difícil escrever um programa *firmware* do que um programa software.

Como podemos usar o conceito de microprogramação para implementar uma unidade de controle? Considere que, para cada micro-operação, tudo o que é permitido para a unidade de controle fazer é gerar um conjunto de sinais de controle. Assim, para cada micro-operação, cada linha de controle que se origina da unidade de controle

Tabela 16.1 Conjunto de instruções de máquina para exemplo de Wilkes

Ordem	Efeito da ordem
An	$C(Acc) + C(n)$ para Acc_1
Sn	$C(Acc) - C(n)$ para Acc_1
Kn	$C(n)$ para Acc_2
Vn	$C(Acc_2) \times C(n)$ para Acc , onde $C(n) \geq 0$
Tn	$C(Acc_1)$ para n , 0 para Acc
Un	$C(Acc_1)$ para n
Rn	$C(Acc) \times 2^{-(n+1)}$ para Acc
Ln	$C(Acc) \times 2^{n+1}$ para Acc
Gn	IF $C(Acc) < 0$, transferir controle para n ; se $C(Acc) \geq 0$, ignorar (isto é, proceder serialmente)
In	Ler próximo caractere do mecanismo de entrada para n
On	Enviar $C(n)$ para mecanismo de saída

Acc = acumulador

Acc_1 = metade mais significativo do acumulador

Acc_2 = metade menos significativo do acumulador

n = localização de armazenamento n

$C(X)$ = conteúdo de X (X = registrador da localização de armazenamento)

está ligada ou desligada. É claro que esta condição pode ser representada por um dígito binário para cada linha de controle. Assim, podemos construir uma palavra de controle onde cada bit represente uma linha de controle. Então, cada micro-operação seria representada por um padrão diferente de 1 e 0 na palavra de controle.

Suponha que façamos uma cadeia de uma sequência de palavras de controle para representar a sequência de micro-operações executadas pela unidade de controle. A seguir, devemos reconhecer que a sequência de micro-operações não é fixa. Às vezes temos um ciclo indireto, às vezes não. Vamos então colocar as nossas palavras de controle em uma memória onde cada palavra possui um endereço único. Adicionamos agora um campo de endereço para cada palavra de controle indicando a posição da próxima palavra de controle a ser executada se uma determinada condição for verdadeira (por exemplo, o bit indireto em uma referência de memória for 1). Além disso, adicionamos alguns bits para especificar a condição.

O resultado é conhecido como uma **microinstrução horizontal** e um exemplo é mostrado na Figura 16.1a. O formato da microinstrução ou palavra de controle está descrito a seguir. Existe um bit para cada linha de controle interna do processador e um bit para cada linha de controle do barramento do sistema. Há um campo de condição indicando a condição em que deve haver um desvio e há um campo com o endereço da microinstrução a ser executada depois que um desvio é tomado. Tal microinstrução é interpretada como segue:

1. Para executar esta microinstrução, ligar todas as linhas de controle indicadas por um bit 1; desligar todas as linhas de controle indicadas por um bit 0. Os sinais de controle resultantes farão com que uma ou mais micro-operações sejam executadas.
2. Se a condição indicada pelos bits de condição for falsa, executar a próxima microinstrução na sequência.
3. Se a condição indicada pelos bits de condição for verdadeira, a próxima microinstrução a ser executada é indicada no campo de endereço.

A Figura 16.2 mostra como essas palavras de controle ou microinstruções poderiam ser arranjadas em uma **memória de controle**. As microinstruções em cada rotina serão executadas sequencialmente. Cada rotina termina com uma instrução de desvio ou salto indicando para onde deve ir a seguir. Existe um ciclo de execução especial cujo único propósito é sinalizar que uma das rotinas de instrução de máquina (AND, ADD e outras) está para ser executada a seguir, dependendo do *opcode* corrente.

A memória de controle da Figura 16.2 é uma descrição concisa da operação completa da unidade de controle. Ela define a sequência de micro-operações para serem executadas durante cada ciclo (busca, indireto, execução, interrupção) e especifica o sequenciamento desses ciclos. Esta notação seria uma maneira útil para documentar o

Figura 16.1 Formatos típicos de microinstruções

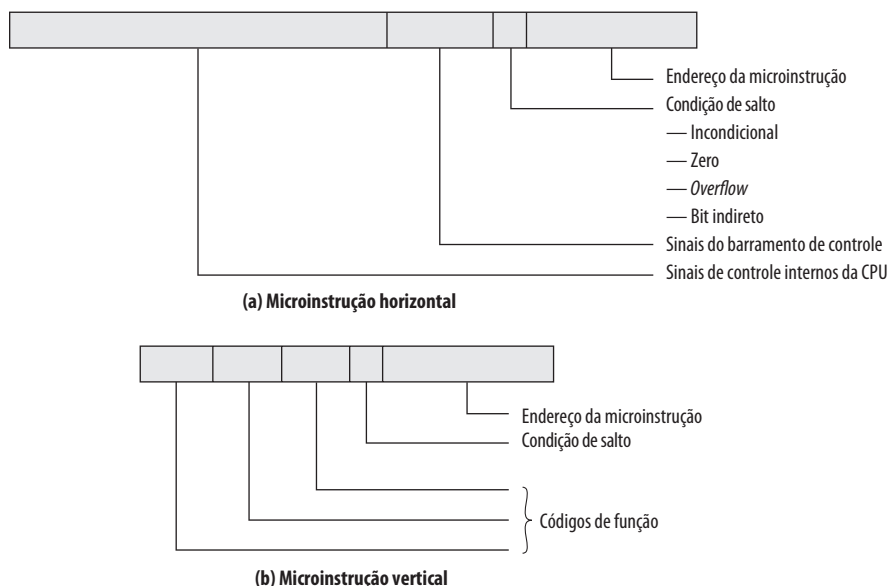
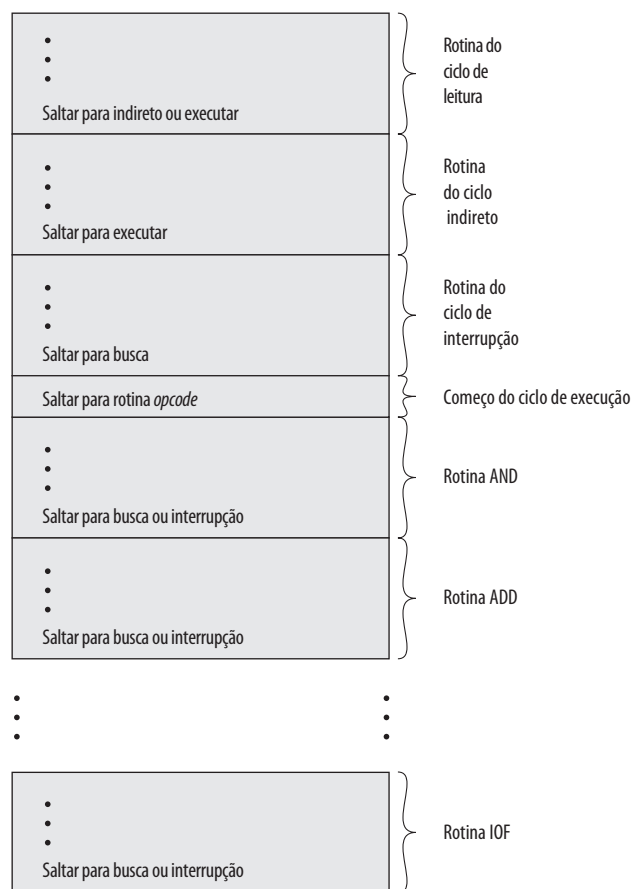


Figura 16.2 Organização da memória de controle

funcionamento de uma unidade de controle de um computador específico, mas ela é mais do que isso. Ela é também uma forma de implementar a unidade de controle.



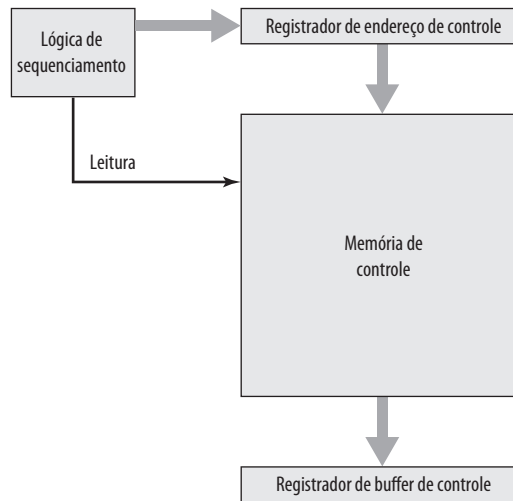
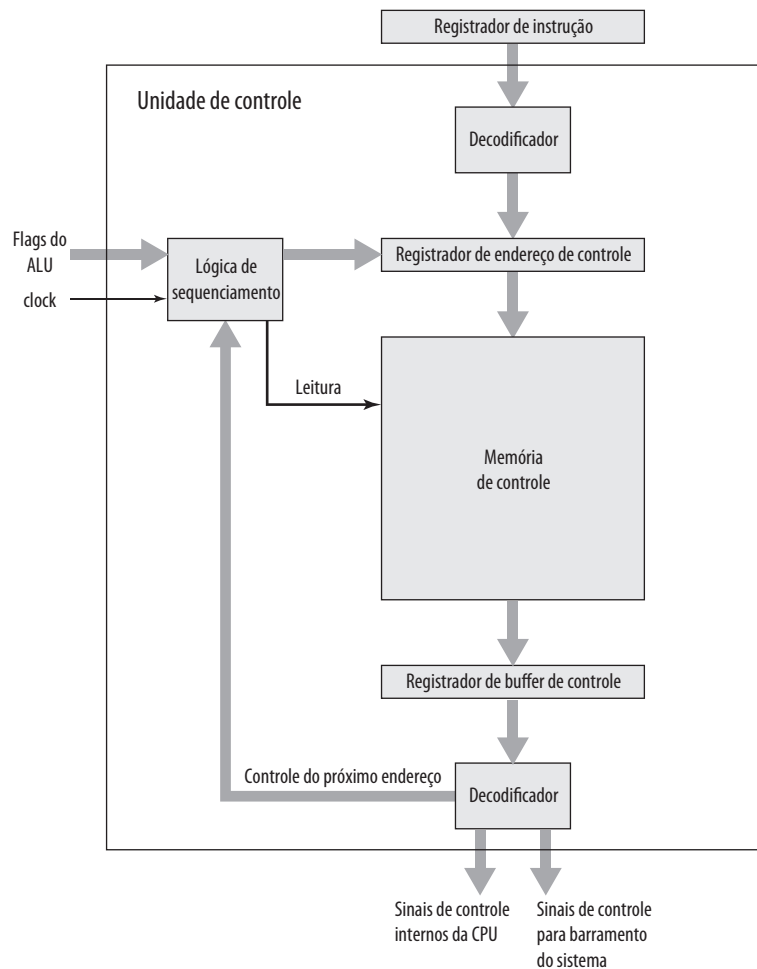
Unidade de controle microprogramada

A memória de controle da Figura 16.2 contém um programa que descreve o comportamento da unidade de controle. Poderíamos implementar a unidade de controle simplesmente executando o programa.

A Figura 16.3 mostra os elementos-chave de tal implementação. O conjunto de microinstruções é armazenado na *memória de controle*. O *registrador de endereço de controle* contém o endereço da próxima microinstrução a ser lida. Quando uma microinstrução é lida a partir da memória de controle, ela é transferida para um *registrador de buffer de controle*. A parte esquerda desse registrador (veja Figura 16.1a) conecta-se às linhas de controle que saem da unidade de controle. Assim, *ler* uma microinstrução a partir da memória de controle é o mesmo que *executar* essa microinstrução. O terceiro elemento mostrado na figura é uma unidade de sequenciamento que carrega o registrador de endereço de controle e emite um comando de leitura.

Vamos analisar esta estrutura em mais detalhes, conforme ilustrado na Figura 16.4. Ao comparar isto com a Figura 16.4, vemos que a unidade de controle ainda tem as mesmas entradas (IR, ALU, flags, clock) e saídas (sinais de controle). A unidade de controle funciona desta forma:

1. Para executar uma instrução, a unidade lógica de sequenciamento emite um comando READ para memória de controle.
2. A palavra cujo endereço é especificado no registrador de endereço de controle é lida para dentro do registrador de buffer de controle.

Figura 16.3 Microarquitetura da unidade de controle**Figura 16.4** Funcionamento da unidade de controle microprogramada

3. O conteúdo do registrador de buffer de controle gera sinais de controle e a informação do próximo endereço para a unidade lógica de sequenciamento.
4. A unidade lógica de sequenciamento carrega um novo endereço no registrador de endereço de controle com base na informação do próximo endereço a partir do registrador de buffer de controle e flags do ALU.

Tudo isto ocorre durante um pulso de clock.

O último passo mencionado precisa ser melhor explicado. Na conclusão de cada microinstrução, a unidade lógica de sequenciamento carrega um novo endereço no registrador de endereço de controle. Dependendo do valor das flags da ALU e do registrador de buffer de controle, uma das três decisões é tomada:

- **Obter a próxima instrução:** adiciona 1 ao registrador de endereço de controle.
- **Saltar para uma nova rotina com base em uma microinstrução de salto:** carregar o campo de endereço do registrador de buffer de controle no registrador de endereço de controle.
- **Saltar para uma rotina de instrução de máquina:** carregar o registrador de endereço de controle com base no *opcode* que está em IR.

A Figura 16.4 mostra dois módulos chamados *decodificadores*. O decodificador superior traduz o *opcode* armazenado em IR para um endereço de controle de memória. O decodificador inferior não é usado para microinstruções horizontais, mas é usado para **microinstruções verticais** (Figura 16.1b). Conforme mencionado, em uma instrução horizontal um código é usado para cada ação a ser executada (por exemplo, $MAR \leftarrow (PC)$) e o decodificador traduz este código em sinais de controle individuais. A vantagem de microinstruções verticais é que elas são mais compactas (menos bits) do que as microinstruções horizontais, a custo de uma pequena de lógica e tempo de atraso condicionais.

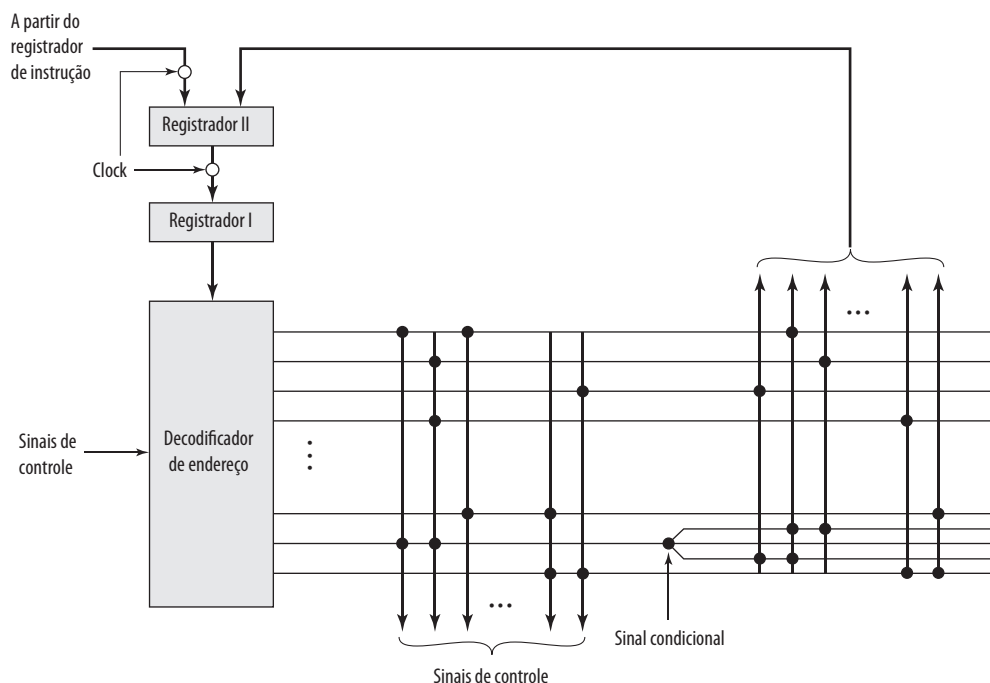


Controle de Wilkes

Conforme mencionamos antes, Wilkes foi o primeiro a propor o uso de uma unidade de controle microprogramada, em 1951 (WILKES, 1951^a). A seguir, esta proposta foi elaborada para um projeto mais detalhado (WILKES e STRINGER, 1953^a). É interessante a análise desta importante proposta.

A configuração proposta por Wilkes é ilustrada na Figura 16.5. O coração do sistema é uma matriz parcialmente preenchida com diodos. Durante um ciclo de máquina, uma linha da matriz é ativada com um pulso. Isto gera

Figura 16.5 A unidade de controle microprogramada de Wilkes



sinais naqueles pontos onde um diodo está presente (indicado por um ponto no diagrama). A primeira parte da linha gera sinais de controle que controlam a operação do processador. A segunda parte gera o endereço da linha a ser estimulada com um pulso no próximo ciclo de máquina. Assim, cada linha da matriz é uma microinstrução e o layout da matriz é a memória de controle.

No início do ciclo, o endereço da linha a ser estimulada com um pulso é contido no Registrador I. Este endereço é a entrada para o decodificador que, quando ativado por um pulso de clock, ativa uma linha da matriz. Dependendo dos sinais de controle, ou o *opcode* no registrador de instrução ou a segunda parte da linha pulsada é passada para Registrador II durante o ciclo. O Registrador II é então chaveado para Registrador I por um pulso de clock. Os pulsos de clock alternados são usados para ativar uma linha da matriz e para transferir do Registrador II para o Registrador I. O arranjo de dois registradores é necessário porque o decodificador é simplesmente um circuito combinatório; com apenas um registrador, a saída se tornaria entrada durante um ciclo, causando uma condição instável.

Este esquema é muito parecido com a abordagem de microprogramação horizontal descrita anteriormente (Figura 16.1a). A principal diferença é: na descrição anterior, o registrador de endereço de controle poderia ser incrementado por 1 para obter a próxima instrução. No esquema de Wilkes, o próximo endereço é contido na microinstrução. Para permitir desvios, uma linha deve conter duas partes do endereço controladas por um sinal condicional (por exemplo, flag), conforme mostrado na figura.

Tendo proposto este esquema, Wilkes fornece um exemplo do seu uso para implementar a unidade de controle de uma máquina simples. Este exemplo, o primeiro projeto conhecido de um processador microprogramado, vale a pena ser repetido aqui porque ele ilustra muitos princípios modernos da microprogramação.

O processador da máquina hipotética inclui os seguintes registradores:

- A multiplicando.
- B acumulador (metade menos significativa).
- C acumulador (metade mais significativa).
- D registrador de deslocamento.

Além disso, existem três registradores e dois flags de 1 bit acessíveis apenas para a unidade de controle. Os registradores são:

- E serve tanto como um registrador de endereço de memória (MAR) ou como um registrador de armazenamento temporário.
- F contador de programa.
- G outro registrador temporário; usado para contagem.

A Tabela 16.1 mostra o conjunto de instruções de máquina para este exemplo. A Tabela 16.2 é o conjunto de microinstruções completo, expresso em forma simbólica, que implementa a unidade de controle. Desta forma, um total de 38 microinstruções é tudo o que é necessário para definir o sistema completamente.

A primeira coluna cheia dá o endereço (número da linha) de cada microinstrução. Aqueles endereços correspondentes aos *opcodes* são rotulados. Assim, quando o *opcode* para a instrução de adição (A) é encontrado, a microinstrução na posição 5 é executada. As colunas 2 e 3 expressam as ações a serem tomadas pela ALU e pela unidade da controle, respectivamente. Cada expressão simbólica deve ser traduzida em um conjunto de sinais de controle (bits da microinstrução). As colunas 4 e 5 especificam o sinal que define as flags. Por exemplo, (1) C_s significa que o flag número 1 é definida pelo bit de sinal do número no registrador C. Se a coluna 5 contém um identificador de flag, então as colunas 6 e 7 contêm dois endereços de microinstrução alternativos para serem usados. Caso contrário, a coluna especifica o endereço da próxima microinstrução a ser obtida.

As instruções de 0 a 4 constituem o ciclo de leitura. A microinstrução 4 apresenta o *opcode* para um decodificador, o qual gera o endereço de uma microinstrução correspondendo à instrução de máquina a ser obtida. O leitor deve ser capaz de deduzir o funcionamento completo da unidade de controle a partir de um estudo cuidadoso da Tabela 16.2.



Vantagens e desvantagens

A principal vantagem do uso da microprogramação para implementar uma unidade de controle é que ela simplifica o projeto da unidade de controle. Assim a implementação fica mais barata e menos propensa a erros. Uma unidade de controle por hardware deve conter uma lógica complexa para sequenciamento por meio de várias micro-operações do ciclo de instrução. Por outro lado, os decodificadores e a unidade de sequenciamento lógico de uma unidade de controle microprogramada tem uma lógica muito simples.

Tabela 16.2 Microinstruções do exemplo de Wilkes

Notação: $A, B, C \dots$ simbolizam vários registradores da unidade aritmética e da unidade de controle de registradores. C para D indica que o chaveamento de circuitos conecta a saída do registrador C para entrada do registrador D ; $(D + A)$ para C indica que o registrador de saída de A é conectado a uma entrada da unidade de adição (a saída de D é permanentemente conectada para outra entrada) e a saída do somador ao registrador C . Um símbolo numérico n entre aspas (por exemplo, ' n ') indica a origem cuja saída é o número n em unidades do dígito menos significativo.

		Unidade aritmética	Unidade de controle de registradores	Flip-flop condicional		Próxima microinstrução	
				Definir	Usar	0	1
	0		F para G e E			1	
	1		(G para '1') para F			2	
	2		Armazenamento para G			3	
	3		G para E			4	
	4		E para decodificador			—	
A	5	C para D				16	
S	6	C para D				17	
H	7	Armazenamento para B				0	
V	8	Armazenamento para A				27	
T	9	C para armazenamento				25	
U	10	C para armazenamento				0	
R	11	B para D	E para G			19	
L	12	C para D	E para G			22	
G	13		E para G	(1) C_5		18	
I	14	Entrada para armazenamento				0	
O	15	Armazenamento para saída				0	
	16	(D + Armazenamento) para C				0	
	17	(D – Armazenamento) para C				0	
	18				1	0	1
	19	D para B (R)*	(G – '1') para E			20	
	20	C para D		(1) E_5		21	
	21	D para C (R)			1	11	0
	22	D para C (L) [†]	(G – '1') para E			23	
	23	B para D		(1) E_5		24	
	24	D para B (L)			1	12	0
	25	'0' para B				26	
	26	B para C				0	
	27	'0' para C	'18' para E			28	
	28	B para D	E para G	(1) B_1		29	
	29	D para B (R)	(G – '1') para E			30	
	30	C para D (R)		(2) E_5	1	31	32
	31	D para C			2	28	33
	32	(D + A) para C			2	28	33
	33	B para D		(1) B_1		34	
	34	D para B (R)				35	
	35	C para D (R)			1	36	37
	36	D para C				0	
	37	(D – A) para C				0	

*Deslocamento para direita. Os circuitos de comutação na unidade aritmética são arranjados de tal forma que o dígito menos significativo do registrador C seja colocado na parte mais significativa do registrador B durante as micro-operações de deslocamento para a direita e o dígito mais significativo do registrador C (dígito de sinal) é repetido (fazendo desta forma a correção para números negativos).

[†]Deslocamento para esquerda. Os circuitos de comutação são arranjados de forma semelhante para passar o dígito mais significativo do registrador B para o menos significativo do registrador C durante as micro-operações de deslocamento para a esquerda.

A principal desvantagem de uma unidade microprogramada é que ela será um pouco mais lenta do que uma unidade por hardware de tecnologia comparável. Apesar disso, a microprogramação é a técnica dominante para implementar unidades de controle em arquiteturas CISC puras, por causa da facilidade de sua implementação. Os processadores RISC, com seu formato de instrução mais simples, normalmente usam unidades de controle por hardware. Analisamos a seguir a abordagem de microprogramação em mais detalhes.



16.2 Sequenciamento de microinstruções

As duas tarefas básicas desempenhadas por uma unidade de controle microprogramada são as seguintes:

- **Sequenciamento de microinstruções:** obter a próxima microinstrução da memória de controle.
- **Execução de microinstruções:** gerar os sinais de controle necessários para executar a microinstrução.

Ao se projetar uma unidade de controle, estas tarefas devem ser consideradas juntas, porque ambas afetam o formato da microinstrução e a temporização da unidade de controle. Nesta seção, nos concentramos no sequenciamento e falamos o mínimo possível sobre questões de formato e temporização. Essas questões são analisadas em mais detalhes na próxima seção.



Considerações sobre projeto

Duas preocupações são envolvidas no projeto de uma técnica de sequenciamento de microinstruções: o tamanho da microinstrução e o tempo de geração do endereço. A primeira preocupação é óbvia; minimizar o tamanho da memória de controle reduz o custo desse componente. A segunda preocupação é simplesmente o desejo de executar as microinstruções o mais rapidamente possível.

Ao executar um microprograma, o endereço da próxima microinstrução a ser executada se encaixa em uma destas categorias:

- Determinado pelo registrador de instrução.
- Próximo endereço sequencial.
- Desvio.

A primeira categoria ocorre apenas uma vez por ciclo de instrução, logo depois que uma instrução é obtida. A segunda categoria é a mais comum na maioria dos projetos. No entanto, o projeto não pode ser otimizado apenas para o acesso sequencial. Desvios, condicionais e incondicionais, são uma parte necessária de um microprograma. Além disso, as sequências de microinstruções tendem a ser curtas: uma de cada três ou quatro microinstruções poderia ser um desvio (SIEWIOREK, BELL e NEWELL, 1982^d). Assim, é importante projetar técnicas compactas e eficientes para desvio de microinstruções.

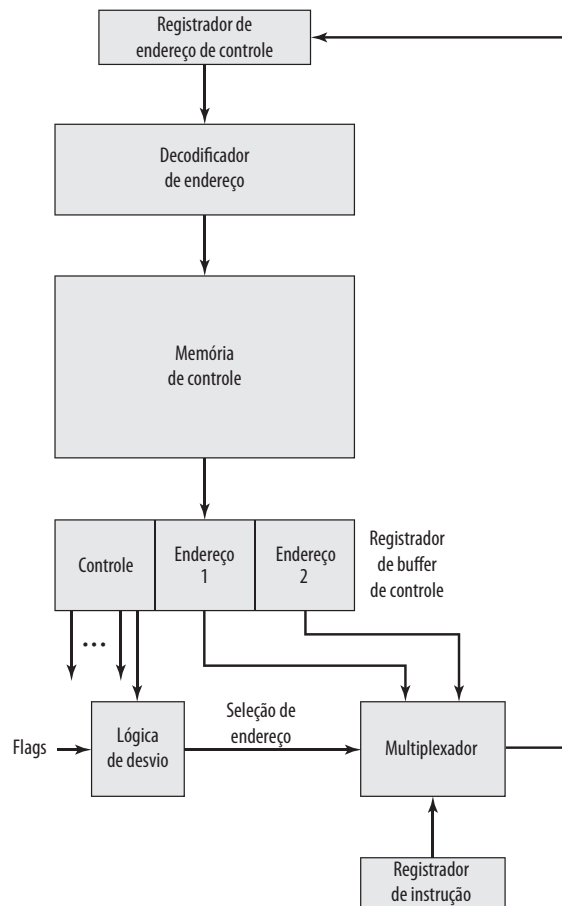


Técnicas de sequenciamento

Com base na microinstrução corrente, nos flags de condição e no conteúdo do registrador de instrução, um endereço de memória de controle deve ser gerado para a próxima microinstrução. Uma grande variedade de técnicas tem sido usada. Podemos agrupá-las em três categorias gerais, conforme ilustrado nas figuras 16.6 até 16.8. Estas categorias são baseadas no formato da informação de endereço na microinstrução:

- Dois campos de endereço.
- Campo de endereço único.
- Formato variável.

A abordagem mais fácil é fornecer dois campos de endereço em cada microinstrução. A Figura 16.6 sugere como essa informação deve ser usada. Um multiplexador existente e serve como destino para os campos de endereço e para o registrador de instrução. Com base em uma entrada de seleção de endereço, o multiplexador transmite o *opcode* ou um dos dois endereços para o registrador de endereço de controle (CAR, do inglês *control address register*). A seguir, o CAR é decodificado para produzir o endereço da próxima microinstrução. Os sinais de seleção de endereço são fornecidos por um módulo de lógica de desvio cuja entrada consiste de flags da unidade de controle mais os bits da parte de controle da microinstrução.

Figura 16.6 Lógica de controle de desvio: dois campos de endereço

Embora a abordagem de dois endereços seja simples, ela requer mais bits dentro da microinstrução do que outras abordagens. Economias podem ser alcançadas com alguma lógica adicional. Uma abordagem comum é ter um campo de endereço único (Figura 16.7). Com esta abordagem, as opções para o próximo endereço são:

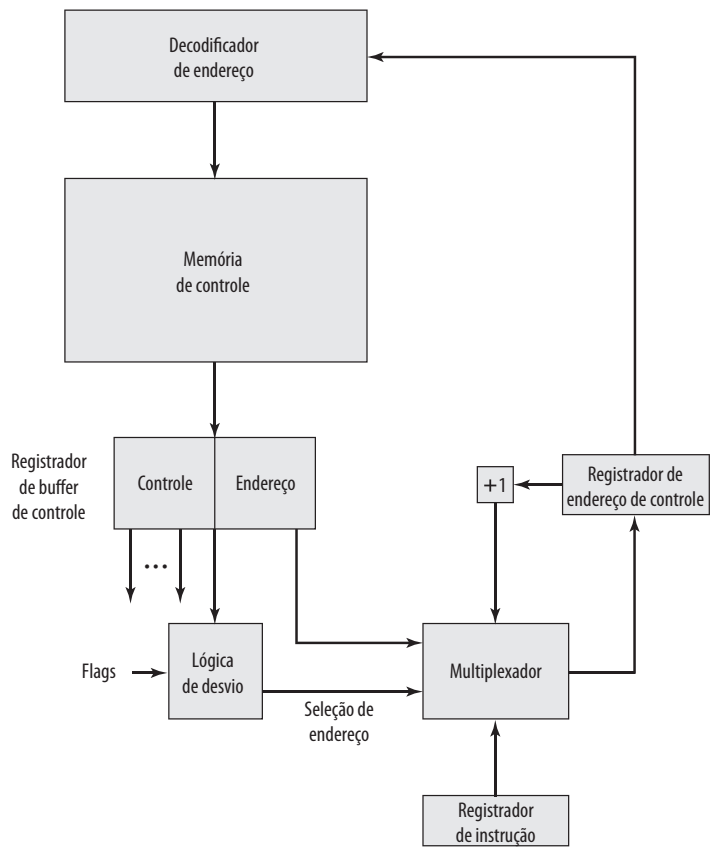
- Campo de endereço.
- Código do registrador de instrução.
- Próximo endereço sequencial.

Os sinais de seleção de endereço determinam qual opção está selecionada. Esta abordagem reduz o número de campos de endereço para 1. Observe, no entanto, que o campo de endereço frequentemente não é usado. Assim, há alguma ineficiência no esquema de codificação da microinstrução.

Outra abordagem é fornecer dois formatos de instrução totalmente diferentes (Figura 16.8). Um bit define qual formato está sendo usado. Em um formato, os bits restantes são usados para ativar sinais de controle. No outro formato, alguns bits conduzem o módulo de lógica de desvio e os bits restantes fornecem o endereço. Com o primeiro formato, o próximo endereço é o próximo endereço sequencial ou um endereço derivado a partir do registrador de instrução. Com o segundo formato, é especificado um desvio condicional ou um incondicional. Uma desvantagem desta abordagem é que um ciclo inteiro é consumido com cada microinstrução de desvio. Com outras abordagens, a geração de endereço ocorre como parte do mesmo ciclo da geração de sinais de controle, minimizando acessos à memória de controle.

As abordagens que acabamos de descrever são gerais. Implementações específicas frequentemente irão envolver uma variação ou uma combinação dessas técnicas.

Figura 16.7 Lógica de controle de desvio: campo de endereço único



Geração de endereços

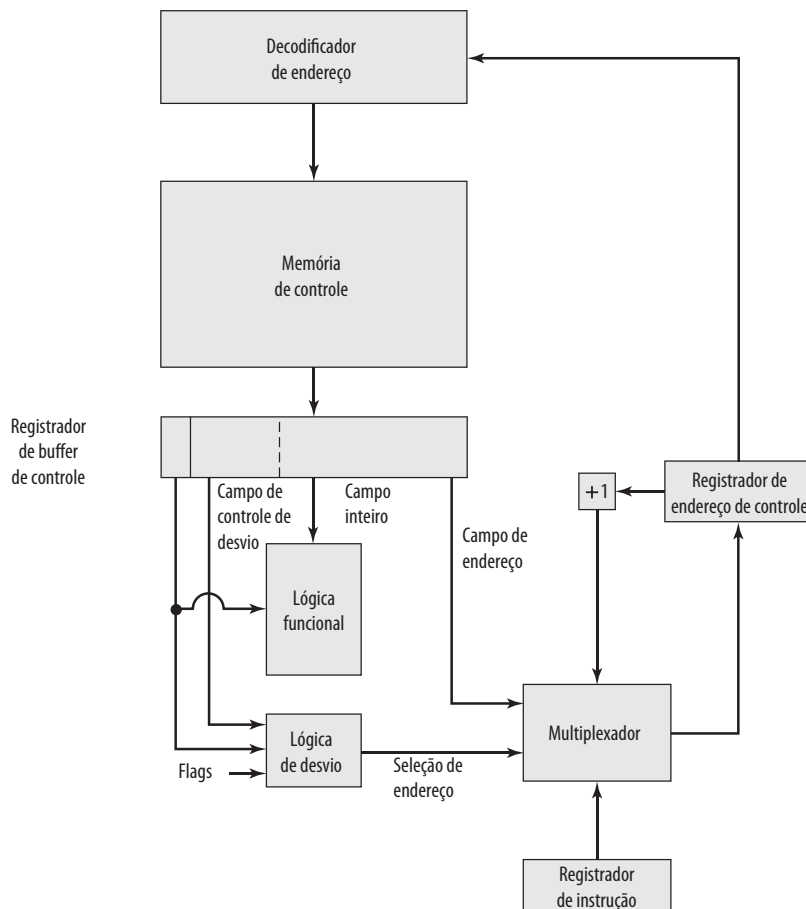
Analizamos o problema de sequenciamento do ponto de vista da consideração de formato e dos requisitos de lógica gerais. Outro ponto de vista é considerar várias maneiras em que o próximo endereço pode ser derivado ou computado.

A Tabela 16.3 mostra várias técnicas de geração de endereços. Elas podem ser divididas em técnicas explícitas, onde o endereço está disponível explicitamente na microinstrução, e técnicas implícitas, que requerem lógica adicional para gerar o endereço.

Nós lidamos basicamente com técnicas explícitas. Com a abordagem de dois campos, dois endereços alternativos estão disponíveis em cada microinstrução. Usando o campo de endereço único ou o formato variável, várias

Tabela 16.3 Técnicas de geração de endereço da microinstrução

Explícita	Implícita
Dois campos	Mapeamento
Desvio incondicional	Adição
Desvio condicional	Controle residual

Figura 16.8 Lógica de controle de desvio: formato variável

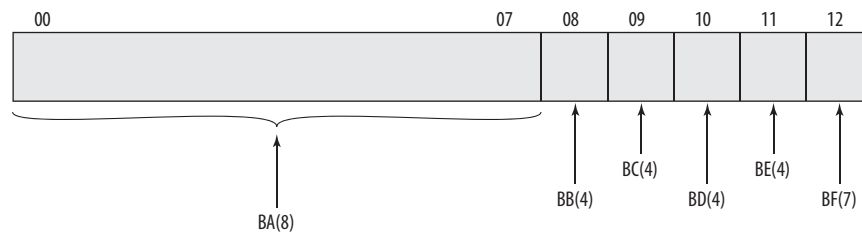
instruções de desvio podem ser implementadas. Uma instrução de desvio condicional depende dos seguintes tipos de informação:

- Flags da ALU.
- Parte do *opcode* ou campo de modo de endereço da instrução de máquina.
- Partes do registrador selecionado, como o bit de sinal.
- Bits de *status* dentro da unidade de controle.

Várias técnicas implícitas também são comumente usadas. Uma delas, o mapeamento, é necessária em quase todos os projetos. A parte *opcode* de uma instrução de máquina deve ser mapeada em um endereço de microinstrução. Isto ocorre apenas uma vez por ciclo de instrução.

Uma técnica implícita comum é a que envolve a combinação ou a adição de duas partes de um endereço para formar o endereço completo. Esta abordagem foi usada na família IBM S/360 (TUCKER, 1967^e) e também em muitos modelos S/370. Usaremos o IBM 3033 como exemplo.

O registrador de endereço de controle do IBM 3033 possui o tamanho de 13 bits e está ilustrado na Figura 16.9. Duas partes do endereço podem ser diferenciadas; 8 bits de ordem mais alta (00-07) normalmente não mudam de um ciclo de microinstrução para outro. Durante a execução de uma microinstrução, estes 8 bits são copiados diretamente de um campo de 8 bits da microinstrução (campo BA) para os 8 bits de ordem mais alta do registrador de endereço de controle. Isto define um bloco de 32 microinstruções na memória de controle. Os 5 bits restantes do registrador de endereço de controle são definidos para especificar o endereço específico da microinstrução a ser obtida. Cada um destes bits é determinado por um campo de 4 bits (exceto um que é um campo de 7 bits) da

Figura 16.9 Registrador de endereço de controle do IBM 3033

microinstrução corrente; o campo especifica a condição para definir o bit correspondente. Por exemplo, um bit no registrador de endereço de controle pode ser definido para 1 ou 0, dependendo de o carry ter acontecido na última operação da ALU.

A abordagem final listada na Tabela 16.3 é chamada de **controle residual**. Esta abordagem envolve o uso de um endereço da microinstrução que foi salvo previamente em armazenamento temporário dentro da unidade de controle. Por exemplo, alguns conjuntos de microinstruções incluem com uma facilidade para sub-rotinas. Um registrador interno ou uma pilha de registradores é usado para guardar os endereços de retorno. Um exemplo desta abordagem é usado em LSI-11, o qual analisaremos agora.



Sequenciamento de microinstruções do LSI-11

O LSI-11 é uma versão de microcomputador de um PDP-11, com os componentes principais do sistema residindo em uma placa única. Ele é implementado usando uma unidade de controle microprogramada (SEBERN, 1976^f).

O LSI-11 faz uso de uma microinstrução de 22 bits e uma memória de controle de palavras de 2K e 22 bits. O endereço da próxima microinstrução é determinado em uma das cinco maneiras:

- **Próximo endereço sequencial:** na ausência de outras instruções, o registrador de endereço de controle da unidade de controle é incrementado por 1.
- **Mapeamento de opcode:** no começo de cada ciclo de instrução, o próximo endereço de microinstrução é determinado pelo *opcode*.
- **Facilidade de subrotina:** explicado a seguir.
- **Testes de interrupção:** certas microinstruções especificam um teste para interrupção. Se uma interrupção ocorre, isso determina o endereço da próxima microinstrução.
- **Desvio:** microinstruções de desvio condicionais e incondicionais são usadas.

Um mecanismo de sub-rotina de um nível é fornecida. Um bit em cada microinstrução é dedicado a esta tarefa. Quando o bit está definido com valor 1, um registrador de retorno de 11 bits é carregado com o conteúdo atualizado do registrador de endereço de controle. Uma microinstrução subsequente que especifica um retorno irá fazer com que o registrador de endereço de controle seja carregado a partir do registrador de retorno.

O retorno é uma forma de instrução de desvio incondicional. Outra forma de desvio incondicional faz com que os bits do registrador de endereço de controle sejam carregados a partir dos 11 bits da microinstrução. A instrução de desvio condicional faz uso de um código de teste de 4 bits dentro da microinstrução. Este código especifica testes de vários códigos condicionais da ALU para determinar a decisão de desvio. Se a condição não for verdadeira, o próximo endereço sequencial é selecionado. Se for verdadeira, os 8 bits de ordem mais baixa do registrador de endereço de controle são carregados a partir dos 8 bits da microinstrução. Isto permite desvios dentro de uma página de memória de 256 palavras.

Como pode ser visto, o LSI-11 inclui uma facilidade de sequenciamento de endereços poderosa dentro da unidade de controle. Isso permite ao microprogramador uma flexibilidade considerável e pode facilitar a tarefa de microprogramação. Por outro lado, esta abordagem requer mais lógica de unidade de controle do que potencialidades mais simples.



16.3 Execução de microinstruções

O ciclo de microinstrução é o evento básico em um processador microprogramado. Cada ciclo é feito de duas partes: busca e execução. A parte de busca é determinada pela geração de um endereço de microinstrução e tratamos disso na seção anterior. Esta seção trata da execução de uma microinstrução.

Lembre-se que o efeito da execução de uma microinstrução é gerar sinais de controle. Alguns desses sinais de controle apontam para dentro do processador. Os sinais restantes vão para o barramento de controle externo ou para outras interfaces externas. Como uma função secundária, o endereço de uma microinstrução é determinado.

A descrição anterior sugere a organização de uma unidade de controle mostrada na Figura 16.10. Esta versão levemente revisada da Figura 16.4 é o foco desta seção. Os principais módulos neste diagrama deveriam estar claros até agora. O módulo lógico de sequenciamento contém a lógica para efetuar as funções discutidas na seção anterior. Ele gera o endereço da próxima microinstrução, usando como entradas o registrador de instrução, os flags da ALU, o registrador de endereço de controle (para incrementar) e o registrador de buffer de controle. O último pode fornecer um endereço real, bits de controle ou ambos. O módulo é controlado por um clock que determina o tempo do ciclo de microinstrução.

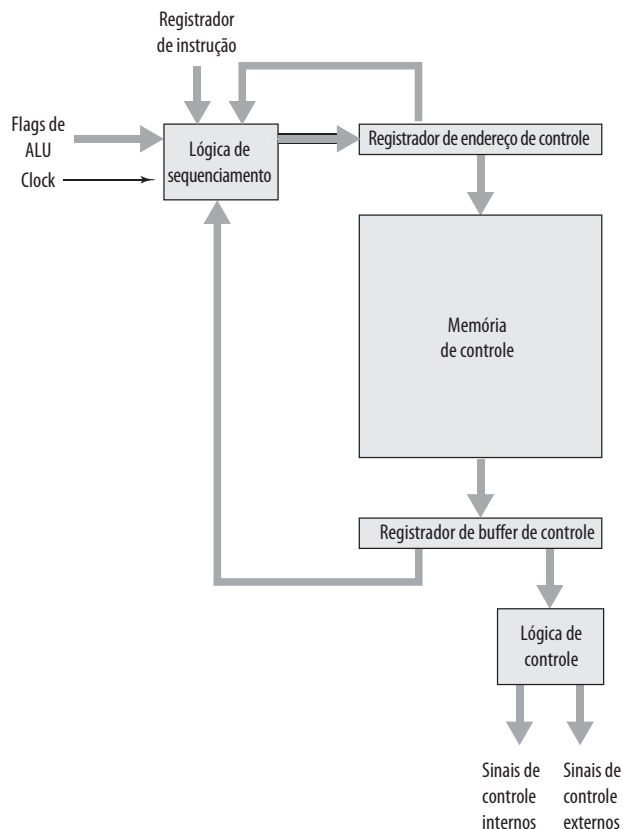
O módulo lógico de controle gera sinais de controle em função de alguns bits da microinstrução. Deve estar claro que o formato e o conteúdo da microinstrução irão determinar a complexidade do módulo lógico de controle.



Taxonomia de microinstruções

As microinstruções podem ser classificadas de várias formas. As diferenças feitas comumente na literatura incluem:

Figura 16.10 Organização da unidade de controle



- Vertical/horizontal.
- Empacotada/não empacotada.
- Microprogramação hard/soft.
- Codificação direta/indireta.

Todas elas têm a ver com o formato da microinstrução. Nenhum destes termos foi usado de forma consistente e precisa na literatura. No entanto, uma análise desses pares de termos serve para esclarecer as alternativas para projeto de microinstruções. Nos próximos parágrafos, analisamos primeiro a principal questão que forma a base de todos esses pares de características e depois analisamos os conceitos sugeridos por cada par.

Na proposta original de Wilkes (1951^a), cada bit de uma microinstrução produz diretamente um sinal de controle ou produz diretamente um bit do próximo endereço. Vimos na seção anterior que esquemas de sequenciamento de endereços mais complexos usando menos bits de microinstrução são possíveis. Esses esquemas requerem um módulo lógico de sequenciamento mais complexo. Um compromisso similar existe para a parte da microinstrução referente aos sinais de controle. Ao codificar a informação de controle e subsequentemente decodificando-a para produzir sinais de controle, os bits da palavra de controle podem ser economizados.

Como essa codificação pode ser feita? Para responder à questão, considere que existe um total de K diferentes sinais de controle internos e externos para serem conduzidos pela unidade de controle. No esquema de Wilkes, K bits da microinstrução seriam dedicados para esse propósito. Isso permite que todas as 2^K combinações possíveis de sinais de controle sejam geradas durante qualquer ciclo de instrução, mas nós podemos fazer melhor que isso se observarmos que nem todas as combinações possíveis serão usadas. Exemplos incluem o seguinte:

- Duas origens não podem ser chaveadas para o mesmo destino (por exemplo, C_2 e C_8 na Figura 16.5).
- Um registrador não pode ser a fonte e ao mesmo tempo destino ao mesmo tempo (por exemplo, C_5 e C_{12} na Figura 16.5).
- Apenas um padrão de sinais de controle pode ser apresentado à ALU ao mesmo tempo.
- Apenas um padrão de sinais de controle pode ser apresentado para o barramento de controle externo ao mesmo tempo.

Então, para um dado processador, todas as possíveis combinações de sinais de controle permitidas poderiam ser listadas, dado algum número $Q < 2^K$ de possibilidades. Estas poderiam ser codificadas com $\log_2 Q$ bits, com $(\log_2 Q) < K$. Esta seria a forma mais compacta de codificação que preserva todas as combinações permitidas de sinais de controle. Na prática, esta forma de codificação não é usada por dois motivos:

- É difícil de programar tanto quanto um esquema de decodificação (Wilkes) puro. Este ponto é discutido logo a seguir.
- Ele requer um módulo lógico de controle complexo e, portanto, lento.

Em vez disso, alguns compromissos são adotados. Existem dois tipos deles:

- São usados mais bits do que estritamente necessário para codificar as combinações possíveis.
- Algumas combinações que são permitidas fisicamente não são possíveis de codificar.

O último tipo de compromisso tem o efeito de reduzir o número de bits. O resultado final, no entanto, é usar mais do que $\log_2 Q$ bits.

Na próxima subseção, vamos discutir técnicas de codificação específicas. O restante desta subseção trata dos efeitos da codificação e dos vários termos usados para descrevê-la.

Com base no anterior, podemos ver que a parte de sinal de controle do formato da microinstrução falha em um espectro. Em uma extremidade, há um bit para cada sinal de controle; em outra extremidade, um formato altamente codificado é usado. A Tabela 16.4 mostra que outras características de uma unidade de controle microprogramada também falham em um espectro e que esses espectros são, em grande parte, determinados pelo espectro do grau de codificação.

O segundo par de itens na tabela é bastante óbvio. O esquema de Wilkes puro requer a maioria dos bits. Também deve estar claro que este extremo representa a visão detalhada do hardware. Cada sinal de controle é controlável individualmente pelo microprogramador. A codificação é feita de maneira a agregar funções ou recursos, para que o microprogramador possa ver o processador de um nível mais alto e menos detalhado. Além disso, a codificação é desenvolvida para facilitar o trabalho de microprogramação. Novamente, deve estar claro que a tarefa de entender e combinar o uso de todos os sinais de controle é difícil. Conforme mencionamos, uma das consequências comuns da codificação é prevenir o uso de algumas combinações que seriam permitidas de outra forma.

Tabela 16.4 Espectro de microinstruções

Características	
Não codificado	Altamente codificado
Muitos bits	Poucos bits
Visão detalhada de hardware	Visão agregada de hardware
Dificuldade de programar	Facilidade de programar
Concorrência totalmente explorada	Concorrência não totalmente explorada
Pequena ou nenhuma lógica de controle	Lógica de controle complexa
Execução rápida	Execução lenta
Desempenho otimizado	Programação otimizada
Terminologia	
Não empacotada	Empacotada
Horizontal	Vertical
Hard	Soft

O parágrafo anterior discute o projeto da microinstrução do ponto de vista do microprogramador. Mas o nível de codificação pode ser visto também a partir dos seus efeitos de hardware. Com um formato puro não codificado, nenhuma ou pouca lógica é necessária; cada bit gera um sinal de controle particular. Conforme são usados os esquemas de codificação mais compactos e mais agregados, uma lógica de decodificação mais complexa é necessária. Isto, por sua vez, pode afetar o desempenho. Mais tempo é necessário para propagar sinais pelas portas do módulo lógico de controle mais complexo. Assim, a execução das microinstruções codificadas pode levar mais tempo do que a execução das não codificadas.

Assim, todas as características listadas na Tabela 16.4 estão dentro de um espectro de estratégias de projeto. Em geral, um projeto que segue o lado esquerdo do espectro tem a intenção de otimizar o desempenho da unidade de controle. Os projetos do lado direito são mais preocupados em otimizar o processo de microprogramação. Na verdade, os conjuntos de instruções próximos do lado direito do espectro se parecem muito com conjuntos de instruções de máquina. Um bom exemplo disso é o projeto do LSI-11, descrito anteriormente neste capítulo. Normalmente, quando o objetivo é simplesmente implementar uma unidade de controle, o projeto tenderá mais para o lado esquerdo do espectro. O projeto do IBM 3033, discutido agora, está nessa categoria. Conforme discutiremos depois, alguns sistemas permitem que vários usuários construam diferentes microprogramas usando a mesma facilidade de microinstruções. Nos últimos exemplos, o projeto estará mais próximo do lado direito do espectro.

Podemos agora lidar com alguma terminologia introduzida anteriormente. A Tabela 16.4 indica como três desses pares de termos se relacionam com o espectro de microinstruções. Basicamente, todos esses pares descrevem a mesma coisa, porém enfatizam características de projeto diferentes.

O grau de empacotamento relaciona-se com o grau de identificação entre uma determinada tarefa de controle e bits específicos de microinstruções. À medida que os bits se tornam mais *empacotados*, certo número de bits contém mais informação. Assim, o empacotamento implica codificação. Os termos *horizontal* e *vertical* referem-se ao tamanho relativo da microinstrução. Siewiorek, Bell e Newell (1982^d) sugerem como regra que as microinstruções verticais possuam o tamanho no intervalo entre 16 e 40 bits e que as microinstruções horizontais possuam o tamanho no intervalo entre 40 e 100 bits. Os termos microprogramação *hard* e *soft* são usados para sugerir o grau de proximidade com os sinais de controle e layout de hardware subjacentes. Microprogramas *hard* são normalmente fixos e dedicados para memória ROM. Microprogramas *soft* são mais mutáveis e são sugestivos da microprogramação do usuário.

O outro par de termos mencionado no início desta subseção refere-se à codificação direta *versus* a indireta, um assunto que analisaremos agora.



Codificação de microinstruções

Na prática, as unidades de controle microprogramadas não são projetadas usando um formato de microinstrução puramente horizontal ou não codificado. Pelo menos algum grau de codificação é usado para reduzir o tamanho da memória de controle e para simplificar a tarefa de microprogramação.

A técnica básica de codificação é ilustrada na Figura 16.11a. A microinstrução é organizada como um conjunto de campos. Cada campo contém um código que, depois de decodificado, ativa um ou mais sinais de controle.

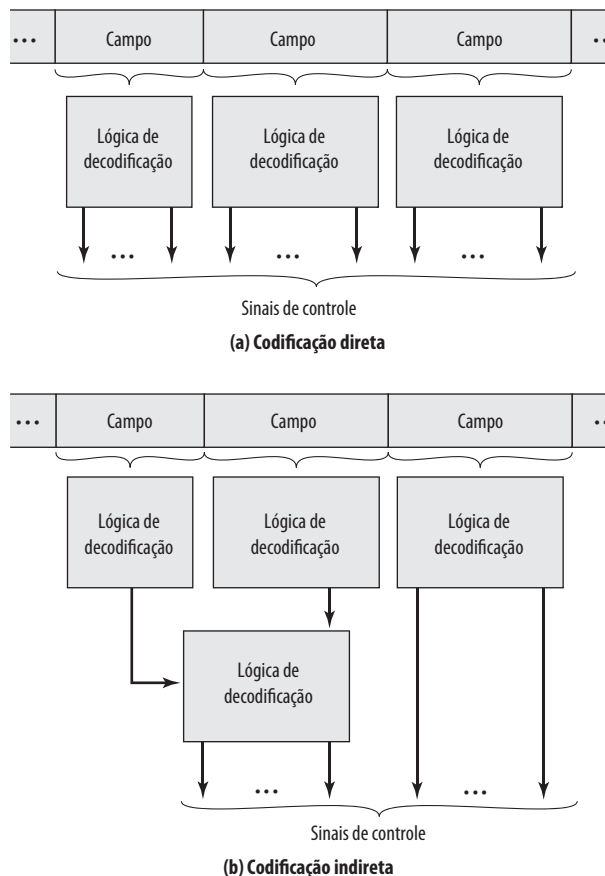
Vamos considerar as implicações deste layout. Quando a microinstrução é executada, cada campo é decodificado e gera sinais de controle. Assim, com N campos, N ações simultâneas são especificadas. Cada ação resulta na ativação de um ou mais sinais de controle. Geralmente, mas nem sempre, queremos projetar o formato de tal forma que cada sinal de controle seja ativado por não mais do que um campo. No entanto, é claro que deve ser possível para cada sinal de controle ser ativado por pelo menos um campo.

Considere agora um campo individual. Um campo que consiste de L bits pode conter um de 2^L códigos, cada um deles podendo ser codificado para um padrão de sinal de controle diferente. Como apenas um código pode aparecer em um campo ao mesmo tempo, os códigos são mutuamente exclusivos e, por isso, as ações que eles causam são mutuamente exclusivas.

O projeto de um formato de microinstrução codificado pode ser definido agora de forma simples:

- Organize o formato em campos independentes. Ou seja, cada campo ilustra um conjunto de ações (padrões de sinais de controle) de tal forma que ações de campos diferentes possam ocorrer simultaneamente.
- Defina cada campo de tal forma que ações alternativas que podem ser especificadas pelo campo sejam mutuamente exclusivas. Ou seja, apenas uma das ações especificadas para um determinado campo pode ocorrer por vez.

Figura 16.11 Codificação da microinstrução



Duas abordagens podem ser adotadas para organizar uma microinstrução codificada em campos: a funcional e a de recursos. O método de *codificação funcional* identifica funções dentro da máquina e define os campos pelo tipo de função. Por exemplo, se várias fontes podem ser usadas para transferir dados para o acumulador, um campo pode ser projetado para este propósito, com cada código especificando uma fonte diferente. A *codificação de recursos* vê a máquina como um conjunto de recursos independentes e dedica um campo para cada um deles (por exemplo, E/S, memória, ALU).

Outro aspecto de codificação é se ela é direta ou indireta (Figura 16.11b). Com codificação indireta, um campo é usado para determinar a interpretação de outro campo. Por exemplo, considere uma ALU que é capaz de efetuar oito operações aritméticas diferentes e oito operações de deslocamento diferentes. Um campo de 1 bit poderia ser usado para indicar se uma operação de deslocamento ou aritmética deve ser usada; um campo de 3 bits indicaria a operação. Esta técnica implica geralmente dois níveis de decodificação, aumentando os atrasos de propagação.

A Figura 16.12 é um exemplo simples destes conceitos. Suponha um processador com um acumulador único e vários registradores internos, como um contador de programa e um registrador temporário para entradas da ALU. A Figura 16.12a mostra um formato altamente vertical. Os três primeiros bits indicam o tipo de operação, os três próximos codificam a operação e dois últimos selecionam um registrador interno. A Figura 16.12b é uma abordagem mais horizontal, embora codificação ainda seja usada. Neste caso, funções diferentes aparecem em campos diferentes.



Execução de microinstruções no LSI-11

O LSI-11 (SEBERN, 1976¹) é um bom exemplo de uma abordagem de microinstrução vertical. Analisamos primeiro a organização da unidade de controle e depois o formato da microinstrução.

ORGANIZAÇÃO DA UNIDADE DE CONTROLE DO LSI-11 O LSI-11 é o primeiro membro da família PDP-11 que foi disponibilizado como um processador de placa única. A placa contém três chips LSI, um barramento interno conhecido como *barramento de microinstruções* (MIB) e alguma lógica adicional para interfaces.

A Figura 16.13 ilustra de uma forma simples a organização do processador LSI-11. Os três chips são chips de dados, controle e armazenamento de controle. O chip de dados contém uma ALU de 8 bits, 26 registradores de 8 bits e armazenamento para vários códigos condicionais. Dezesseis dos registradores são usados para implementar oito registradores de uso geral de 16 bits de PDP-11. Outros incluem uma palavra de *status* de programa, registrador de endereço de memória (MAR) e registrador de buffer de memória. Como a ALU lida com apenas 8 bits ao mesmo tempo, duas passagens por ela são necessárias para implementar uma operação aritmética de 16 bits do PDP-11. Isto é controlado pelo microprograma.

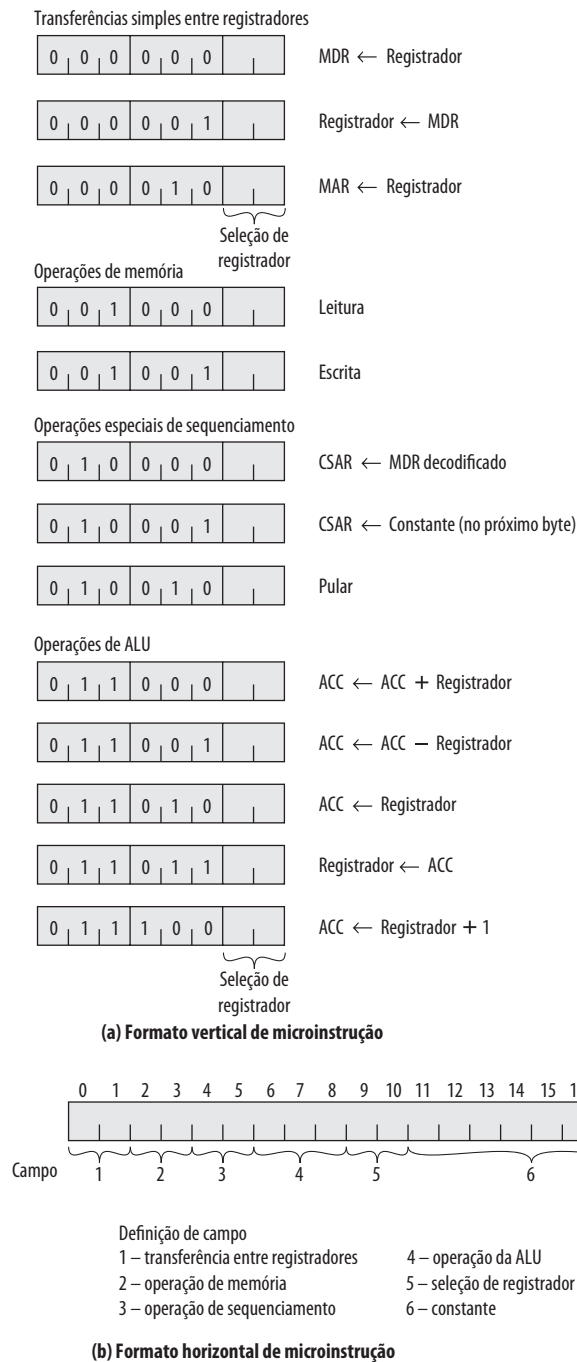
O chip ou os chips de armazenamento de controle contêm a memória de controle com largura de 22 bits. O chip de controle contém a lógica para sequenciamento e execução de microinstruções. Ele contém o registrador de endereço de controle, registrador de controle de dados e uma cópia do registrador de instrução de máquina.

O MIB junta todos os componentes. Durante a leitura da microinstrução, o chip de controle gera um endereço de 11 bits em MIB. O armazenamento de controle é acessado, produzindo uma microinstrução de 22 bits que é colocada em MIB. Os 16 bits de ordem mais baixa vão para o chip de dados, enquanto os 18 bits de ordem baixa vão para o chip de controle. Os 4 bits de ordem mais alta controlam as funções especiais da placa do processador.

A Figura 16.14 fornece uma visão ainda mais simples, porém mais detalhada, da unidade de controle de LSI-11: a figura ignora limites individuais dos chips. O esquema de sequenciamento de endereços descrito na Seção 16.2 é implementado em dois módulos. O controle geral de sequência é fornecido pelo módulo de controle de sequência microprogramado, o qual é capaz de incrementar o registrador de endereço da microinstrução e efetuar desvios incondicionais. Outras formas de calcular o endereço são realizadas por um vetor de tradução separado. Este é um circuito combinatório que gera um endereço com base na microinstrução, na instrução de máquina, no contador de programa de microinstrução e em um registrador de interrupção.

O vetor de tradução atua nas seguintes situações:

- Quando o *opcode* é usado para determinar o início de uma microrrotina.
- Em tempos apropriados, bits de modo de endereço da microinstrução são testados para efetuar endereçamento apropriado.

Figura 16.12 Formatos de microinstrução alternativos para um máquina simples

- Quando condições de interrupção são testadas periodicamente.
- Quando microinstruções de desvios condicionais são avaliadas.

FORMATO DA MICROINSTRUÇÃO DO LSI-11 O LSI-11 usa um formato extremamente vertical de microinstrução com tamanho de apenas 22 bits. O conjunto de microinstruções assemelha-se muito ao conjunto de instruções de máquina do PDP-11 que ele implementa. Este projeto tem a intenção de otimizar o desempenho da unidade de controle dentro das restrições de um projeto vertical, facilmente programado. A Tabela 16.5 mostra algumas das microinstruções de LSI-11.

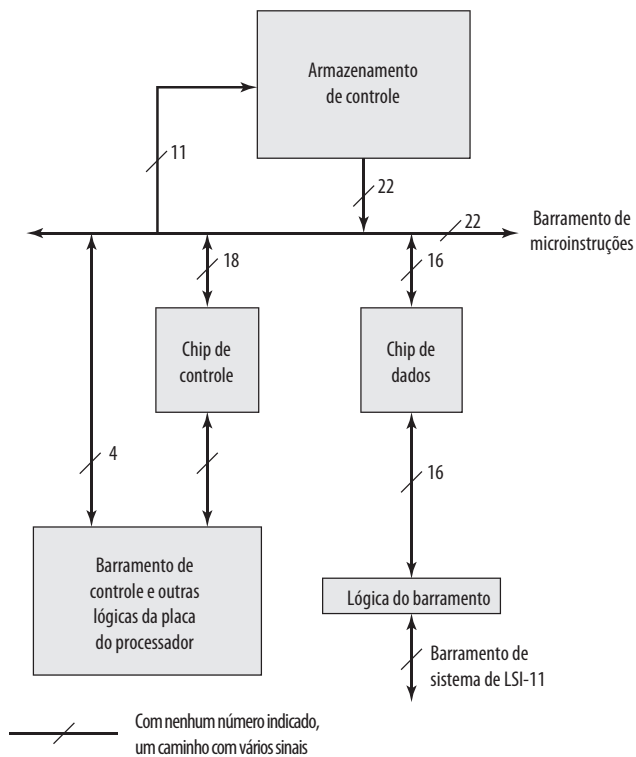
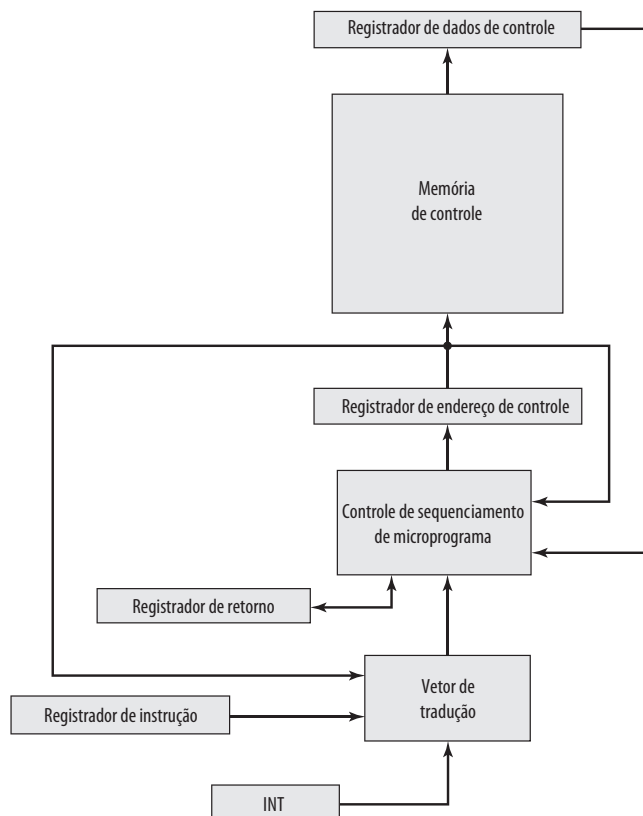
Figura 16.13 Diagrama de blocos simplificado do processador LSI-11**Figura 16.14** Organização da unidade de controle de LSI-11

Tabela 16.5 Algumas microinstruções de LSI-11

Operações aritméticas	Operações gerais
Adicionar palavra (byte, literal)	MOV palavra (byte)
Testar palavra (byte, literal)	Salto
Incrementar palavra (byte) por 1	Retorno
Incrementar palavra (byte) por 2	Salto condicional
Negar palavra (byte)	Ativar (desativar) flags
Incrementar (decrementar) byte condicionalmente	Carregar G baixo
Adicionar palavra (byte) condicionalmente	Condicional MOV palavra (byte)
Adicionar palavra (byte) com carry	Operações de Entrada/Saída
Adicionar dígitos condicionalmente	
Subtrair palavra (byte)	Entrada palavra (byte)
Comparar palavra (byte, literal)	Entrada palavra de status (byte)
Subtrair palavra (byte) com carry	Ler
Decrementar palavra (byte) por 1	Escrever
Operações lógicas	Ler (escrever) e incrementar palavra (byte) por 1
	Ler (escrever) e incrementar palavra (byte) por 2
AND palavra (byte, literal)	Reconhecimento de leitura (escrita)
Testar palavra (byte)	Saída de palavra (byte, <i>status</i>)
OR palavra (byte)	
OR-exclusivo palavra (byte)	
Bit limpar palavra (byte)	
Deslocar palavra (byte) para direita (esquerda) com (sem) carry	
Complementar palavra (byte)	

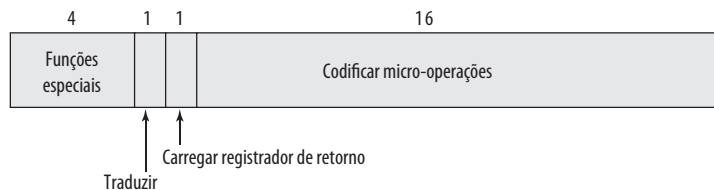
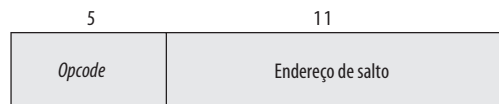
A Figura 16.15 mostra o formato da microinstrução LSI-11 de 22 bits. Os 4 bits de ordem mais alta controlam funções especiais da placa do processador. O bit de tradução habilita que o vetor de tradução verifique interrupções pendentes. O bit do registrador de leitura de retorno é usado no final de uma microrrotina para fazer com que o endereço da próxima microinstrução seja carregado a partir do registrador de retorno.

Os 16 bits restantes são usados para micro-operações altamente codificadas. O formato se parece com uma instrução de máquina, com um *opcode* de tamanho variável e um ou mais operandos.

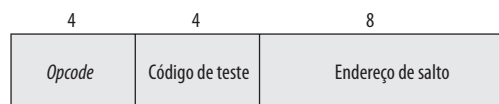


Execução de microinstruções no IBM 3033

A memória de controle padrão do IBM 3033 consiste de palavras de 4K. A primeira parte delas (0000-07FF) contém microinstruções de 108 bits, enquanto o restante (0800-0FFF) é usado para armazenar microinstruções de 126 bits. O formato é ilustrado na Figura 16.16. Embora este seja um formato mais horizontal, a codificação é muito usada. Os principais campos desse formato são resumidos na Tabela 16.6.

Figura 16.15 Formato da microinstrução LSI-11**(a) Formato da microinstrução LSI-11 completa**

Formato da microinstrução de salto incondicional



Formato da microinstrução de salto condicional



Formato da microinstrução literal



Formato da microinstrução de salto de registrador

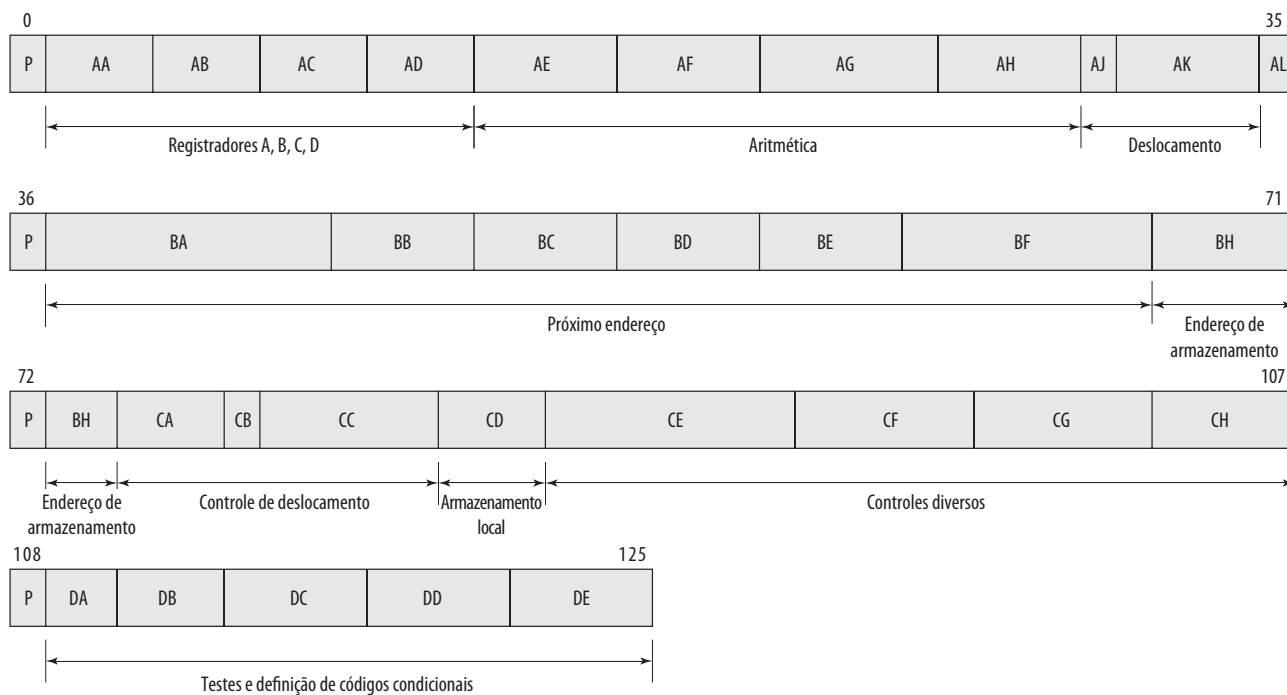
(b) Formato da parte codificada da microinstrução de LSI-11**Figura 16.16** Formato da microinstrução de IBM 3033

Tabela 16.6 Campos de controle da microinstrução do IBM 3033

Campos de controle de ALU	
AA(3)	Carregar registrador A a partir de um dos registradores de dados
AB(3)	Carregar registrador B a partir de um dos registradores de dados
AC(3)	Carregar registrador C a partir de um dos registradores de dados
AD(3)	Carregar registrador D a partir de um dos registradores de dados
AE(4)	Direciona bits especificados A para ALU
AF(4)	Direciona bits especificados B para ALU
AG(5)	Especifica operação aritmética da ALU para entrada A
AH(4)	Especifica operação aritmética da ALU para entrada B
AJ(1)	Especifica entrada D ou B para ALU do lado B
AK(4)	Direciona saída aritmética para o deslocador
CA(3)	Carrega registrador F
CB(1)	Ativa deslocador
CC(5)	Especifica funções lógicas e de carry
CE(7)	Especifica a quantidade de deslocamento
Campos para sequenciamento de desvios	
AL(1)	Termina operação e executa desvio
BA(8)	Ativa bits de ordem mais alta (00-07) do registrador de endereço de controle
BB(4)	Especifica a condição para ativar o bit 8 do registrador de endereço de controle
BC(4)	Especifica a condição para ativar o bit 9 do registrador de endereço de controle
BD(4)	Especifica a condição para ativar o bit 10 do registrador de endereço de controle
BE(4)	Especifica a condição para ativar o bit 11 do registrador de endereço de controle
BF(7)	Especifica a condição para ativar o bit 12 do registrador de endereço de controle

A ALU opera com entradas provenientes de quatro registradores dedicados e não visíveis ao usuário, A, B, C e D. O formato da microinstrução contém campos para carregar esses registradores a partir dos registradores visíveis ao usuário, efetuar uma função da ALU e especificar um registrador visível ao usuário para armazenar o resultado. Há também campos para carregar e armazenar dados entre registradores e memória.

O mecanismo de sequenciamento para IBM 3033 foi discutido na Seção 16.2.

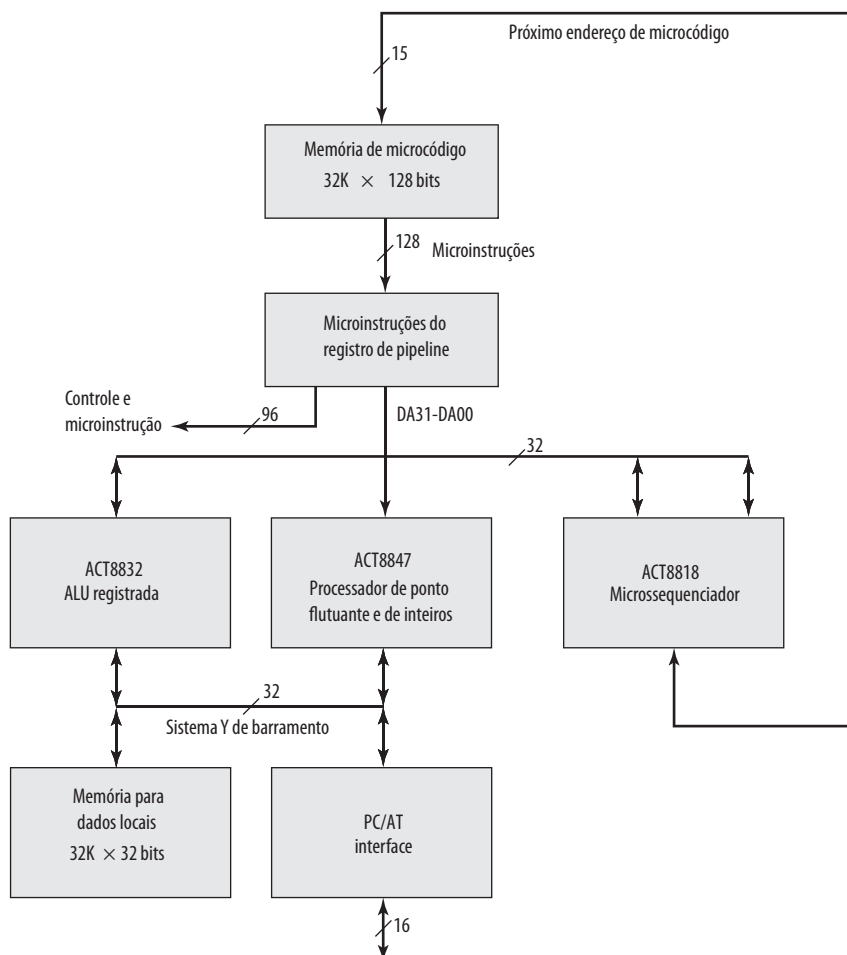


16.4 TI 8800

A *Texas Instruments 8800 Software Development Board* (SDB) é uma placa de computador de 32 bits microprogramável. O sistema possui um armazenamento de controle que pode ser escrito, implementado em RAM em vez de ROM. Tal sistema não alcança a velocidade ou a densidade de um sistema microprogramado com um armazenamento de controle ROM. No entanto, ele é útil para desenvolver protótipos e para fins educacionais.

O 8800 SDB consiste dos componentes a seguir (Figura 16.17):

- Memória de microcódigo.
- Microsequenciador.

Figura 16.17 Diagrama de blocos do TI 8800

- ALU de 32 bits.
- Processador de ponto flutuante e de inteiros.
- Memória para dados locais.

Dois barramentos ligam os componentes internos do sistema. O barramento DA fornece dados a partir do campo de dados da microinstrução para ALU, para o processador de ponto flutuante ou para o microsequeenciador. No último caso, os dados consistem de um endereço para ser usado para uma instrução de desvio. O barramento também pode ser usado pela ALU ou microsequeenciador para fornecer dados para outros componentes. O barramento Y do sistema conecta a ALU e o processador de ponto flutuante com memória local e com módulos externos por meio de interface PC.

A placa se encaixa em um computador compatível com padrão IBM PC. O computador fornece uma plataforma adequada para a montagem e depuração do microcódigo.



Formato da microinstrução

O formato da microinstrução do 8800 consiste de 128 bits separados em 30 campos funcionais, conforme indicado na Tabela 16.7. Cada campo consiste de um ou mais bits e os campos são agrupados em cinco principais categorias:

- Controle da placa.
- Chip de processador 8847 de ponto flutuante e de inteiros.

Tabela 16.7 Formato da microinstrução de TI 8800

Número do campo	Número de bits	Descrição
Controle da placa		
1	5	Seleciona entrada do código condicional
2	1	Habilita/desabilita sinal externo de requisição de E/S
3	2	Habilita/desabilita operações de leitura/escrita em memória de dados locais
4	1	Carrega <i>status</i> /não carrega <i>status</i>
5	2	Determina a unidade que tem controle barramento Y
6	2	Determina a unidade que tem controle barramento DA
Chip de processador 8847 de ponto flutuante e de inteiros		
7	1	Controle do registrador C: usar, não usar clock
8	1	Seleciona bits mais ou menos significativos para barramento Y
9	1	Fonte de dados do registrador C: ALU, multiplexador
10	4	Seleciona modo IEEE ou FAST para ALU e MUL
11	8	Seleciona fontes para operandos de dados: registradores RA, registradores RB, registrador P, registrador S, registrador C
12	1	Controle do registrador RB: usar, não usar clock
13	1	Controle do registrador RA: usar, não usar clock
14	2	Confirmação da fonte de dados
15	2	Habilita/desabilita registradores do pipeline
16	11	Função 8847 de ALU
ALU 8832 registrada		
17	2	Habilitar/desabilitar escuta de dados de saída para registrador selecionado: metade mais significativa, metade menos significativa
18	2	Seleciona fonte de dados do arquivo de registradores: barramento DA, barramento DB, saída ALU Y MUX, barramento Y de sistema
19	3	Modificador de instrução de deslocamento
20	1	Passagem: forçar, não forçar
21	2	Define modo de configuração de ALU: 32, 16 ou 8 bits.
22	2	Seleciona entrada para multiplexador S: arquivo de registradores, barramento DB, registrador MQ
23	1	Seleciona entrada para multiplexador R: arquivo de registradores, barramento DA
24	6	Seleciona registrador no banco C para escrita
25	6	Seleciona registrador no banco B para escrita
26	6	Seleciona registrador no banco A para escrita
27	8	Função de ALU
Microsequeenciador 8818		
28	12	Sinais de controle de entrada para 8818
Campo de dados WCS		
29	16	Bits mais significativos do campo de dados de armazenamento de controle
30	16	Bits menos significativos do campo de dados de armazenamento de controle

- ALU 8832 com registradores.
- Microsequeenciador 8818.
- Campo de dados WCS.

Conforme indicado na Figura 16.17, os 32 bits do campo de dados WCS são alimentados no barramento DA para serem fornecidos como dados para a ALU, o processador de ponto flutuante ou o microsequeenciador. Outros

96 bits (campos 1-27) da microinstrução são sinais de controle alimentados diretamente para o módulo apropriado. Para simplificar, essas outras conexões não são mostradas na Figura 16.17.

Os primeiros seis campos tratam das operações que pertencem ao controle da placa, em vez de controlar um componente individual. As operações de controle incluem:

- Selecionar códigos condicionais para controle do sequenciador. O primeiro bit do campo 1 indica se o flag de condição deve ser definida para 1 ou 0 e os 4 bits restantes indicam qual flag deve ser definida.
- Enviar uma requisição de E/S para PC/AT.
- Habilitar operações de leitura/escrita em memória de dados locais.
- Determinar a unidade que detém o controle barramento Y do sistema. Um dos quatro dispositivos anexos ao barramento (Figura 16.17) é selecionado.

Os últimos 32 bits são o campo de dados que contém a informação específica para uma determinada microinstrução.

Os campos restantes da microinstrução são discutidos melhor dentro do contexto do dispositivo que eles controlam. No restante desta seção, analisamos o microssequenciador e a ALU com registradores. A unidade de ponto flutuante não introduz nenhum conceito novo e é omitida.



Microsssequenciador

A função principal do microssequenciador 8818 é gerar o endereço da próxima microinstrução para o microprograma. Este endereço de 15 bits é fornecido para memória do microcódigo (Figura 16.17).

O próximo endereço pode ser selecionado a partir de uma das cinco origens:

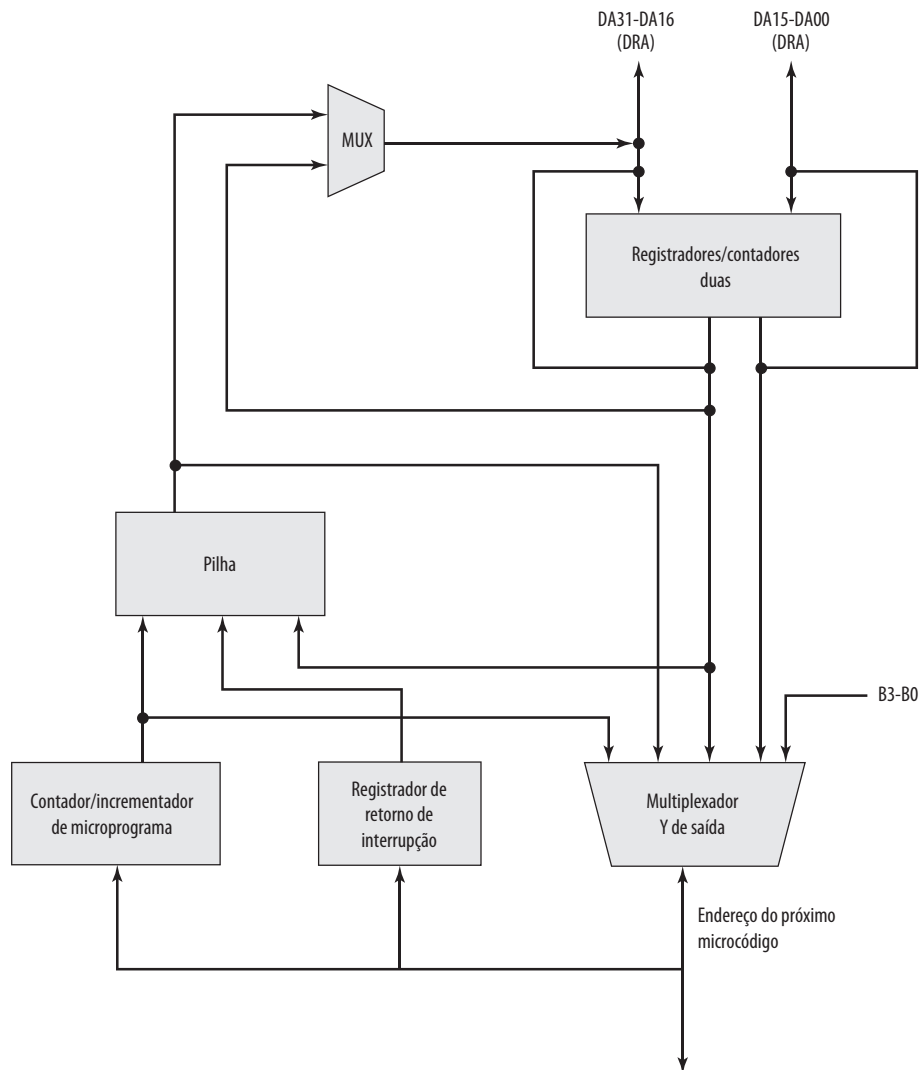
1. O registrador contador de microprograma (MPC), usado para repetir (reutilizar o mesmo endereço) e continuar (incrementar endereço por 1) instruções.
2. A pilha, que suporta chamadas de sub-rotinas do microprograma assim como laços iterativos e retornos das interrupções.
3. Portas DRA e DRB que fornecem dois caminhos adicionais a partir do hardware externo pelos quais os endereços do microprograma podem ser gerados. Estas duas portas são conectadas aos 16 bits mais e menos significativos do barramento DA, respectivamente. Isto permite que o microssequenciador obtenha o endereço da próxima instrução a partir do campo de dados WCS da microinstrução atual ou a partir de um resultado calculado pela ALU.
4. Contadores de registradores RCA e RCB, os quais podem ser usados para armazenamento de endereços adicionais.
5. Uma entrada externa para porta bidirecional Y para suportar interrupções externas.

A Figura 16.18 é um diagrama de blocos lógico de 8818. O dispositivo consiste dos seguintes grupos funcionais principais:

- Um contador de microprograma (MPC) de 16 bits consistindo de um registrador e um incrementador.
- Dois registradores contadores, RCA e RCB, para contar laços e iterações, armazenar endereços dos desvios e conduzir dispositivos externos.
- Uma pilha de 65 palavras de 16 bits que possibilita chamadas de sub-rotinas dos programas e interrupções.
- Um registrador de retorno de interrupção e saída Y possibilitam o processamento de interrupções em nível de microinstruções.
- Um multiplexador Y de saída pelo qual o próximo endereço pode ser selecionado de RCA, RCB, barramentos externos DRA e DRB ou pilha.

REGISTRADORES/CONTADORES Os registradores RCA e RCB podem ser carregados do barramento DA, a partir da microinstrução corrente ou a partir da saída da ALU. Os valores podem ser usados como contadores para controlar o fluxo de execução e podem ser decrementados automaticamente quando acessados. Os valores podem ser usados também como endereços das microinstruções para serem fornecidos ao multiplexador Y de saída. O controle independente de ambos os registradores durante um único ciclo de microinstrução é suportado, exceto decremento simultâneo de ambos os registradores.

PILHA A pilha permite vários níveis de chamadas ou interrupções aninhadas e pode ser usada para suportar desvios e laços. Tenha em mente que estas operações referem-se à unidade de controle, não ao processador como todo, e que os endereços envolvidos são os das microinstruções na memória de controle.

Figura 16.18 Microsequeenciador de TI 8818

Seis operações de pilha são possíveis:

1. Limpar, o que define o ponteiro da pilha para zero, esvaziando a pilha.
2. Desempilhar, o que decremente o ponteiro da pilha.
3. Empilhar, o que coloca o conteúdo de MPC, registrador de retorno da interrupção, ou do barramento DRA na pilha e incrementa o ponteiro da pilha.
4. Ler, que torna o endereço indicado pelo ponteiro de leitura disponível no multiplexador Y de saída.
5. Manter, o que faz o endereço do ponteiro da pilha permanecer inalterado.
6. Carregar ponteiro da pilha, que carrega os sete bits menos significativos de DRA no ponteiro da pilha.

CONTROLE DO MICROSEQUENCIADOR O microsequeenciador é controlado principalmente pelo campo de 12 bits da microinstrução atual, campo 28 (Tabela 16.7). Este campo consiste de seguintes subcampos:

- **OSEL (1 bit):** seleciona saída. Determina qual valor será colocado na saída do multiplexador que alimenta o barramento DRA (canto superior esquerdo da Figura 16.18). A saída é selecionada para vir da pilha ou do registrador RCA. O DRA então serve como entrada para o multiplexador Y de saída ou para registrador RCA.

- **SELDR (1 bit):** seleciona barramento DR. Se definido para 1, este bit seleciona barramento DA externo como entrada para barramentos DRA/DRB. Se definido para 0, seleciona a saída do multiplexador DRA para barramento DRA (controlado por OSEL) e conteúdo de RCB para barramento DRB.
- **ZEROIN (1 bit):** usado para indicar um desvio condicional. O comportamento do microssequenciador dependerá então do código condicional selecionado no campo 1 (Tabela 16.7).
- **RC2-RC0 (3 bits):** controles de registradores. Estes bits determinam a mudança no conteúdo dos registradores RCA e RCB. Cada registrador pode permanecer o mesmo, ser decrementado ou ser carregado a partir dos barramentos DRA/DRB.
- **S2-S0 (3 bits):** controles da pilha. Estes bits determinam qual operação de pilha será executada.
- **MUX2-MUX0:** controles da saída. Estes bits, juntos com o código condicional quando usado, controlam o multiplexador Y de saída e, portanto, o endereço da próxima microinstrução. O multiplexador pode selecionar as suas saídas a partir da pilha, DRA, DRB ou MPC.

Estes bits podem ser definidos individualmente pelo programador. No entanto, normalmente isso não é feito. Em vez disso, os programadores usam mnemônicos que equivalem aos padrões de bits que seriam necessários normalmente. A Tabela 16.8 lista 15 mnemônicos para o campo 28. Um montador de microcódigo os converte em padrões de bits apropriados.

Como um exemplo, a instrução INC88181 é usada para fazer com que a próxima microinstrução na sequência seja selecionada, se o código condicional selecionado atualmente for 1. Da Tabela 16.8 temos

INC88181 = 000000111110

o que é decodificado diretamente para

- **OSEL = 0:** seleciona RCA como saída de DRA saída de MUX; neste caso, a seleção é irrelevante.
- **SELDR = 0:** conforme definido anteriormente; novamente, isto é irrelevante para esta instrução.
- **ZEROIN = 0:** combinado com o valor para MUX indica que nenhum desvio deve ser tomado.
- **R = 000:** retém o valor atual de RA e RC.
- **S = 111:** retém o estado atual da pilha.
- **MUX = 110:** escolhe MPC quando código condicional = 1, DRA quando código condicional = 0.

Tabela 16.8 Bits da microinstrução do microssequenciador TI 8818 (Campo 28)

Mnemônico	Valor	Descrição
RST8818	00000000110	Instrução de reinicialização
BRA88181	011000111000	Desvia para instrução DRA
BRA88180	010000111110	Desvia para instrução DRA
INC88181	000000111110	Instrução de continuação
INC88180	000000000000	Instrução de continuação
CAL88181	010000110000	Salta para sub-rotina no endereço especificado por DRA
CAL88180	010000101110	Salta para sub-rotina no endereço especificado por DRA
RET8818	000000011010	Retorno de sub-rotina
PUSH8818	000000110111	Empilha endereço de retorno da interrupção na pilha
POP8818	100000010000	Retorno de interrupção
LOADDRA	000010111110	Carrega contador DRA do barramento DA
LOADDRB	000110111110	Carrega contador DRB do barramento DA
LOADDRAB	000110111100	Carrega DRA/DRB
DECRDRA	010001111100	Decrementa contador DRA e desvia se não for zero
DECRDRB	010101111100	Decrementa contador DRB e desvia se não for zero



ALU com registradores

A 8832 é uma ALU de 32 bits com 64 registradores que pode ser configurada para operar como quatro ALUs de 8 bits, duas ALUs de 16 bits ou uma única ALU de 32 bits.

Ela é controlada pelos 39 bits que constituem os campos de 17 a 27 da microinstrução (Tabela 16.7); estes são fornecidos para a ALU como sinais de controle. Além disso, conforme indicado na Figura 16.17, a 8832 possui conexões externas com o barramento DA de 32 bits e o barramento Y do sistema de 32 bits. As entradas de DA podem ser fornecidas simultaneamente como dados de entrada para arquivo de registradores de 64 palavras e para módulo lógico da ALU. A entrada do barramento Y do sistema é fornecida para módulo lógico da ALU. Os resultados das operações da ALU e de deslocamento são saídas para barramento DA ou barramento Y do sistema. Os resultados podem ser também alimentados de volta para o banco interno de registradores.

Três portas com endereço de 6 bits permitem que uma leitura de dois operandos e uma escrita de operando sejam executadas dentro do banco de registradores simultaneamente. Um deslocador MQ e um registrador MQ podem também ser configurados para funcionar independentemente para implementar operações de deslocamento de precisão dupla de 8, 16 e 32 bits.

Os campos de 17 até 26 de cada microinstrução controlam o caminho em que os dados fluem dentro de 8832 e entre o 8832 e o ambiente externo. Os campos são os seguintes:

17. **Habilitar escrita.** Estes dois bits especificam escrita de 32 bits, ou 16 bits mais significantes ou 16 bits menos significativos ou não escrevem no banco de registradores. O registrador de destino é definido pelo campo 24.
18. **Selecionar origem de dados do arquivo de registradores.** Se uma escrita está para ocorrer no arquivo de registradores, estes dois bits especificam a origem: barramento DA, barramento DB, saída de ALU ou barramento Y do banco sistema.
19. **Modificador da instrução de deslocamento.** Especifica opções relacionadas ao fornecimento de bits finais de preenchimento e bits de leitura que são deslocados durante as instruções de deslocamento.
20. **Carry in.** Este bit indica se um bit é passado na ALU para esta operação.
21. **Modo de configuração da ALU.** A 8832 pode ser configurada para operar como uma ALU de 32 bits, duas ALUs de 16 bits ou quatro ALUs de 8 bits.
22. **Entrada S.** Entradas do módulo lógico de ALU são fornecidas por dois multiplexadores internos conhecidos como multiplexadores S e R. Este campo seleciona a entrada para ser fornecida pelo multiplexador S: arquivo de registradores, barramento DB ou registrador MQ. Registrador de origem é definido pelo campo 25.
23. **Entrada R.** Seleciona entrada para ser fornecida pelo multiplexador R: arquivo de registradores ou barramento DA.
24. **Registrador destino.** Endereço ou registrador no arquivo de registradores para ser usado para operando destino.
25. **Registrador de origem.** Endereço ou registrador no arquivo de registradores para ser usado para operando fonte, fornecido pelo multiplexador S.
26. **Registrador fonte.** Endereço ou registrador no arquivo de registradores para ser usado para operando de origem, fornecido pelo multiplexador R.

Finalmente, o campo 27 é um *opcode* de 8 bits que especifica a função aritmética ou lógica a ser executada pela ALU. A Tabela 16.9 lista operações diferentes que podem ser executadas.

Como um exemplo de codificação usada para especificar campos de 17 até 27, considere a instrução para adicionar conteúdo do registrador 1 para registrador 2 e colocar resultado no registrador 3. A instrução simbólica é

CONT11[17], WELH, SELRYFYMX, [24], R3, R2, R1, PASS + ADD

O montador traduzirá isso em padrão de bits apropriado. Os componentes individuais da instrução podem ser descritos a seguir:

- CONT11 é a instrução NOP básica.
- Campo [17] é alterado para WELH (habilitar escrita, baixa e alta), para que seja feita uma escrita em um registrador de 32 bits seja da saída Y para a ALU.
- Campo [18] é alterado para SELRYFYMX para selecionar o retorno da saída ALU Y MUX.

Tabela 16.9 Campo de instrução da ALU 8832 registrada (Campo 27)

Grupo 1		Função
ADD	H#01	$R + S + C_n$
SUBR	H#02	$(\text{NOT } R) + S + C_n$
SUBS	H#03	$R = (\text{NOT } S) + C_n$
INSC	H#04	$S + C_n$
INCNS	H#05	$(\text{NOT } S) + C_n$
INCR	H#06	$R + C_n$
INCNR	H#07	$(\text{NOT } R) + C_n$
XOR	H#09	$R \text{ XOR } S$
AND	H#0A	$R \text{ AND } S$
OR	H#0B	$R \text{ OR } S$
NAND	H#0C	$R \text{ NAND } S$
NOR	H#0D	$R \text{ NOR } S$
ANDNR	H#0E	$(\text{NOT } R) \text{ AND } S$
Grupo 2		Função
SRA	H#00	Deslocamento aritmético à direita com precisão única
SRAD	H#10	Deslocamento aritmético à direita com precisão dupla
SRL	H#20	Deslocamento lógico à direita com precisão única
SRLD	H#30	Deslocamento lógico à direita com precisão dupla
SLA	H#40	Deslocamento aritmético à esquerda com precisão única
SLAD	H#50	Deslocamento aritmético à esquerda com precisão dupla
SLC	H#60	Deslocamento circular à esquerda com precisão única
SLCD	H#70	Deslocamento circular à esquerda com precisão dupla
SRC	H#80	Deslocamento circular à direita com precisão única
SRCD	H#90	Deslocamento circular à direita com precisão dupla
MQSRA	H#A0	Deslocamento aritmético à direita do registrador MQ
MQSRL	H#B0	MQ para deslocamento lógico à direita do registrador MQ
MQSLL	H#C0	MQ para deslocamento lógico à esquerda do registrador MQ
MQSLC	H#D0	Deslocamento circular à esquerda do registrador MQ
LOADMQ	H#E0	Carregar registrador MQ
PASS	H#F0	Passar ALU para Y (sem operação de deslocamento)
Grupo 3		Função
SET1	H#08	Ativar bit 1
SET0	H#18	Ativar bit 0
TB1	H#28	Testar bit 1
TB0	H#38	Testar bit 0
ABS	H#48	Valor absoluto
SMTC	H#58	Sinal e magnitude/complemento de dois
ADDI	H#68	Adicionar imediato
SUBI	H#78	Subtrair imediato
BADD	H#88	Adicionar byte R para S
BSUBS	H#98	Subtrair byte S de R
BSUBR	H#A8	Subtrair byte R de S
BINCS	H#B8	Incrementar byte S
BINCNS	H#C8	Incrementar byte S negativo

(Continua)

Tabela 16.9 Campo de instrução da ALU 8832 registrada (Campo 27) (continuação)

Grupo 3		Função
BXOR	H#D8	XOR de byte R e S
BAND	H#E8	AND de byte R e S
BOR	H#F8	OR de byte R e S
Grupo 4		Função
CRC	H#00	Acumular caractere com redundância cíclica
SEL	H#10	Selecionar S ou R
SNORM	H#20	Normalizar tamanho simples
DNORM	H#30	Normalizar tamanho duplo
DIVRF	H#40	Ajustar resto de divisão
SDIVQF	H#50	Fixar quociente de divisão com sinal
SMULI	H#60	Iteração de multiplicação com sinal
SMULT	H#70	Término de multiplicação com sinal
SDIVIN	H#80	Inicializar divisão com sinal
SDIVIS	H#90	Começar divisão com sinal
SDIVI	H#A0	Iterar divisão com sinal
UDIVIS	H#B0	Iniciar divisão sem sinal
UDIVI	H#C0	Iterar divisão sem sinal
UMULI	H#D0	Iterar multiplicação sem sinal
SDIVIT	H#E0	Terminar divisão com sinal
UDIVIT	H#F0	Terminar divisão sem sinal
Grupo 5		Função
LOADFF	H#0F	Carregar flip-flops da divisão/BCD
CLR	H#1F	Limpar
DUMPFF	H#5F	Saída dos flip-flops da divisão/BCD
BCDBIN	H#7F	BCD para binário
EX3BC	H#8F	Correção de excesso de-3 de palavra
EX3C	H#9F	Correção de excesso de-3 de palavra
SDIVO	H#AF	Teste de overflow de divisão com sinal
BINEX3	H#DF	Binário para excesso – 3
NOP32	H#FF	Nenhuma operação

- Campo [24] é alterado para definir o registrador R3 como registrador destino.
- Campo [25] é alterado para definir o registrador R2 como um dos registradores de origem.
- Campo [26] é alterado para definir o registrador R1 como um dos registradores de origem.
- Campo [27] é alterado para especificar uma operação ADD da ALU. A instrução do deslocador de ALU é PASS; assim, a saída de ALU não é deslocada pelo deslocador.

Vários comentários podem ser feitos a respeito da notação simbólica. Não é necessário especificar o número do campo para campos consecutivos. Ou seja,

CONT11[17], WELH, [18], SELRFYMX

pode ser escrito como

CONT11[17], WELH, SELRFYMX

porque SELRFYMX está no campo 18.

As instruções da ALU do Grupo 1 da Tabela 16.9 devem ser sempre usadas em conjunto com Grupo 2. As instruções de ALU do Grupo 3 até 5 não devem ser usadas com Grupo 2.



16.5 Leitura recomendada

Há vários livros dedicados à microprogramação. Talvez o mais compreensivo seja Lynch (1993^a). Segee e Field (1991^h) apresentam os fundamentos de microcódigos e projeto de sistemas microcodificados por meio de um projeto passo a passo de um processador simples de 16 bits. Carter (1996ⁱ) também apresenta os conceitos básicos usando uma máquina simples. Parker e Hamblen (1989^j) e Texas Instruments (1990^k) fornecem uma descrição detalhada de *TI 8800 Software Development Board*.

Vassiliadis, Wong e Cotozana (2003^l) discutem a evolução do uso de microcódigo no projeto de computadores e seu *status* atual.

Principais termos, perguntas de revisão e problemas

Principais termos

Memória de controle	Codificação de microinstruções	Unidade de controle microprogramada
Palavra de controle	Execução de microinstruções	Linguagem de microprogramação
<i>Firmware</i>	Sequenciamento de microinstruções	Microprogramação soft
Microprogramação hard	Microinstruções	Microinstrução não empacotada
Microprogramação horizontal	Microprograma	Microinstrução vertical

Perguntas de revisão

- 16.1 Qual é a diferença entre uma implementação por hardware e uma implementação microprogramada de uma unidade de controle?
- 16.2 Como é interpretada uma microinstrução horizontal?
- 16.3 Qual é o propósito de uma memória de controle?
- 16.4 Qual é a sequência típica na execução de uma microinstrução horizontal?
- 16.5 Qual é a diferença entre microinstruções horizontais e verticais?
- 16.6 Quais são tarefas básicas executadas por uma unidade de controle microprogramada?
- 16.7 Qual é a diferença entre microinstruções empacotadas e não empacotadas?
- 16.8 Qual é a diferença entre programação hard e soft?
- 16.9 Qual é a diferença entre codificação funcional e de recursos?
- 16.10 Enumere algumas aplicações comuns da microprogramação.

Problemas

- 16.1 Descreva a implementação da instrução múltipla na máquina hipotética projetada por Wilkes. Use narrativa e um fluxograma.
- 16.2 Suponha um conjunto de microinstruções que inclui uma microinstrução com a seguinte forma simbólica:

IF ($AC_0 = 1$) THEN $CAR \leftarrow (C_{0-6})$ ELSE $CAR \leftarrow (CAR) + 1$

onde AC_0 é o bit de sinal do acumulador e C_{0-6} são primeiros sete bits da microinstrução. Usando esta microinstrução, escreva um microprograma que implementa uma instrução de máquina Branch Register Minus (BRM) que desvia se AC for negativo. Suponha que os bits de C_1 até C_n da microinstrução especificam um conjunto paralelo de micro-operações. Expresse o programa simbolicamente.

- 16.3 Um processador simples possui quatro fases principais para o seu ciclo de instrução: busca, indireto, execução e interrupção. Dois flags de 1 bit definem a fase atual em uma implementação por hardware.

- a. Por que estes flags são necessários?
 - b. Por que eles não são necessários em uma unidade de controle microprogramada?
- 16.4** Considere a unidade de controle da Figura 16.7. Suponha que a memória de controle tenha um tamanho de 24 bits. A parte de controle do formato da microinstrução é dividida em dois campos. Um campo de micro-operação de 13 bits que especifica as micro-operações a serem efetuadas. Um campo de seleção de endereço que especifica uma condição, com base em flags, que causará um desvio de microinstrução. Existem oito flags.
- a. Quantos bits há no campo de seleção de endereço?
 - b. Quantos bits há no campo de endereço?
 - c. Qual é o tamanho da memória de controle?
- 16.5** Como pode ser feito o desvio incondicional sob circunstâncias do problema anterior? Como o desvio pode ser evitado; ou seja, descreva uma microinstrução que não especifica nenhum desvio, condicional ou incondicional.
- 16.6** Queremos fornecer 8 palavras de controle para cada rotina de instrução de máquina. Os *opcodes* da instrução de máquina têm 5 bits e a memória de controle possui 1.024 palavras. Sugira um mapeamento do registrador de instrução para registrador de endereço de controle.
- 16.7** Um formato de microinstrução codificado é usado. Mostre como um campo de micro-operação de 9 bits pode ser dividido em subcampos para especificar 46 ações diferentes.
- 16.8** Um processador tem 16 registradores, uma ALU com 16 funções lógicas e 16 aritméticas e um deslocador com 8 operações, todos conectados por um barramento interno do processador. Projete um formato de microinstrução para especificar várias micro-operações para o processador.

Referências

- a WILKES, M. "The best way to design an automatic calculating machine". *Proceedings, Manchester University Computer Inaugural Conference*, jul. 1951.
- b HILL, R. "Stored logic programming and applications". *Datamation*, fev. 1964.
- c WILKES, M. e STRINGER, J. "Microprogramming and the design of the control circuits in an electronic digital computer". *Proceedings of the Cambridge Philosophical Society*, abr. 1953. Reimpresso em Siewiorek, Bell e Newell, 1982.
- d SIEWIOREK, D.; BELL, C; e NEWELL, A. *Computer Structures: Principles and Examples*. Nova York: McGraw-Hill, 1982.
- e TUCKER, S. "Microprogram control for System/360". *IBM Systems Journal*, No. 4, 1967.
- f SEBERN, M. "A Minicomputer-compatible microcomputer system: The DEC LSI-11". *Proceedings of the IEEE*, jun. 1976.
- g LYNCH, M. *Microprogrammed state machine design*. Boca Raton, FL: CRC Press, 1993.
- h SEGEE, B. e FIELD, J. *Microprogramming and computer architecture*. Nova York: Wiley, 1991.
- i CARTER, J. *Microprocessor architecture and microprogramming*. Upper Saddle River, NJ: Prentice Hall, 1996.
- j PARKER, A. e HAMBLIN, J. *An introduction to microprogramming with exercises designed for the Texas Instruments SN74ACT8800 Software Development Board*. Dallas, TX: Texas Instruments, 1989.
- k TEXAS INSTRUMENTS INC. *SN74ACT880 Family Data Manual*. SCSS006C, 1990.
- l VASSILIADIS, S.; WONG, S. e COTOFA, S. "Microcode processing: positioning and directions". *IEEE Micro*, jul./ago. de 2003.



Organização paralela

ASSUNTOS DA PARTE 5

A parte final do livro analisa uma importante área em constante crescimento que é a organização paralela. Em uma organização paralela, várias unidades de processamento cooperam para executar aplicações. Enquanto um processador superescalar explora as oportunidades para execução paralela em nível de instruções, uma organização de processamento paralelo procura um nível mais abrangente de paralelismo, um que possibilite que o trabalho seja feito em paralelo e de forma cooperativa por vários processadores. Uma série de questões vem à tona com tais organizações. Por exemplo, se múltiplos processadores, cada um com sua cache, compartilham acesso à mesma memória, então alguns mecanismos, em hardware ou em software, devem ser empregados para garantir que todos os processadores compartilhem uma imagem válida da memória principal: isto é conhecido como problema de coerência de cache. Esta e outras questões de projeto são analisadas na Parte 5.

MAPA DA PARTE 5

Capítulo 17 Processamento paralelo

O Capítulo 17 fornece uma visão sobre as considerações do processamento paralelo. Depois, o capítulo analisa três abordagens para organizar vários processadores: multiprocessadores simétricos (SMP, do inglês *symmetric multiprocessor*), *clusters* e máquinas de acesso não uniforme à memória (NUMA, do inglês *nonuniform memory access*). O SMP e os *clusters* são duas maneiras mais comuns de organizar múltiplos processadores para melhorar o desempenho e a disponibilidade. Sistemas NUMA são um conceito mais novo que ainda não atingiu um sucesso comercial grande, mas que se mostra bastante promissor. Finalmente, o Capítulo 17 analisa uma organização especial conhecida como processador vetorial.

Capítulo 18 Computadores multicore

Um computador multicore é um chip de computador que contém mais do que um processador (núcleo). Chips com vários núcleos possibilitam aumento maior na capacidade computacional quando comparados a uma única capacidade computacional feita continuamente para executar mais rapidamente. O Capítulo 18 analisa algumas questões fundamentais de projeto dos computadores de múltiplos núcleos e fornece exemplos das arquiteturas Intel x86 e ARM.